

**Konzeption und Implementierung eines KI-gestützten
GitLab-Assistenten zur Aufbereitung von Software-Tickets**

Bachelorarbeit

des Studienganges **Informatik**

an der Dualen Hochschule Baden-Württemberg **Stuttgart**

von

Nico Werner

—

Bearbeitungszeitraum	12 Wochen
Matrikelnummer, Kurs	3043145, INF23E
Dualer Partner	Drees & Sommer, Stuttgart
Betreuer*in des Dualen Partners	Denis Günther
Wissenschaftliche Betreuung	Andreas Wickenhäuser

Ehrenwörtliche Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe und diese Arbeit bei keiner anderen Prüfung mit gleichem oder vergleichbarem Inhalt vorgelegt habe und diese bislang nicht veröffentlicht wurde. Des Weiteren versichere ich, dass die eingereichte elektronische Fassung mit der gedruckten Ausfertigung übereinstimmt.

Ort, Datum

Unterschrift

Kurzfassung

Software-Tickets sind in Entwicklungsteams der gemeinsame Ort für fachlichen Bedarf, Umsetzung und Prüfung. In der Praxis fehlen dort häufig entscheidende Angaben. Was wichtig ist, steht oft nicht in der Beschreibung, sondern verteilt in späteren Kommentaren. Bevor die Entwicklung beginnen kann, muss jemand diesen Verlauf von Hand zusammenführen, Widersprüche klären und festhalten, woran der Erfolg erkennbar sein soll. Das verzögert die Arbeit und macht Übergaben schwer nachvollziehbar.

Diese Arbeit geht von genau diesem Problem aus. Die Idee ist ein Assistent, der dort hilft, wo Tickets entstehen: in GitLab. Wird er in einem Ticket angesprochen, liest er den bisherigen Verlauf. Fehlen wichtige Angaben, stellt er gezielte Rückfragen. Sind die Angaben ausreichend, bietet er einen gegliederten Entwurf an. Das Ticket selbst ändert er nicht. Erst wenn ein Mensch den Entwurf prüft und ausdrücklich freigibt, wird die Beschreibung überschrieben.

Dev-Assist setzt diese Idee als Prototyp um. Ein dokumentierter Lauf in GitLab zeigt den Ablauf: Aus einer knappen Meldung entstehen Rückfragen und nach den Antworten ein strukturiertes Ticket. Sechs Personen aus Entwicklung, Qualitätssicherung, Product Ownership und Betrieb bewerteten drei echte Beispieltickets. Sie hielten den aufbereiteten Text für besser geeignet als den jeweiligen Ausgangstext. Das bleibt ein kleines Expertenurteil, kein Nachweis von Zeitgewinn oder besserer Ticketqualität im Alltag. Mehrere Anforderungen an sichere Auslösung und Freigabe sind im gebauten Stand noch nicht erfüllt. Der kontrollierte Ablauf ist als Ziel begründet, vollständig nachgewiesen ist er damit nicht.

Abstract

Software tickets are where development teams record what should be built or fixed, and how the result will be checked. In practice they are often incomplete. Important details are missing or scattered across the description and later comments. Before implementation can start, someone has to gather this information by hand, resolve contradictions, and record how success will be recognised. That delays work and makes handovers hard to follow.

This thesis starts from that problem. The idea is an assistant that helps where tickets are created: in GitLab. When it is mentioned in a ticket, it reads the conversation so far. If important information is missing, it asks targeted questions. If the information is sufficient, it offers a structured draft. It does not change the ticket itself. Only after a person reviews the draft and explicitly approves it is the description overwritten.

Dev-Assist implements this idea as a prototype. A documented GitLab run shows the flow: a brief report leads to questions and, after the answers, a structured ticket. Six people in development, quality assurance, product ownership, and operations rated three real example tickets. They judged the prepared text more suitable than the original. That is a small expert judgement, not evidence of time saved or better tickets in everyday work. Several requirements for safe triggering and release are not yet met in the built version. The controlled flow is justified as a goal; it is not fully demonstrated.

Inhaltsverzeichnis

Ehrenwörtliche Erklärung	I
Kurzfassung	II
Abstract	III
Abbildungsverzeichnis	VII
Tabellenverzeichnis	VIII
Abkürzungsverzeichnis	IX
1. Einleitung	1
1.1. Ziel und Vorgehen	2
1.2. Aufbau und Abgrenzung	3
2. Umgebung und Stand der Technik	4
2.1. GitLab als vorgefundene Umgebung	4
2.1.1. Entwicklungsplattform	4
2.1.2. Issues und Kommentarverlauf	4
2.1.3. Webhooks	4
2.2. Requirements Engineering und Ticketqualität	5
2.2.1. Empirische Befunde zu User Stories und Bug Reports	5
2.3. Large Language Models und agentenbasierte Assistenz	7
2.4. Automatisierte Unterstützung und aktuelle LLM-Forschung	7
2.4.1. Regelbasierte und hybride Werkzeuge	7
2.4.2. Empirische Arbeiten zu LLMs im Requirements Engineering	8
2.5. Risiken generativer Assistenz	11
2.6. Zwischenfazit	11
3. Lösungsidee und Anforderungen	13
3.1. Ausgangsprozess	13
3.2. Qualitätsmodell für Ticketvorschläge	13
3.3. Zielprozess	15
3.4. Stakeholder und Priorisierung	15
3.5. Funktionale Anforderungen	16
3.6. Nicht-funktionale Anforderungen	17
3.7. Herkunft und Nachverfolgbarkeit	18
3.8. Abgrenzung und Zwischenfazit	19

4. Konzeption der Systemarchitektur	21
4.1. Systemkontext und Verantwortlichkeiten	21
4.2. Zentrale Entwurfsentscheidungen	23
4.3. Analyse- und Publish-Pfad	24
4.4. Was das Modell ausgeben darf	26
4.5. Sicherheits- und Testbarkeitsziel	27
4.6. Zwischenfazit	27
5. Implementierung	28
5.1. Was eine Person im Issue sieht	28
5.2. Projektaufbau	33
5.3. Webhook, Aktivierung und Kommando	34
5.4. Analyse, Schema und sichtbare Antwort	35
5.5. Publish	37
5.6. Lokale Tests	37
6. Qualitätssicherung und Evaluation	38
6.1. Was geprüft wurde – und was nicht	38
6.2. Technische Befunde	41
6.2.1. Ereignisannahme und Zugriffsschutz	41
6.2.2. Modellgrenze und Ausgabequalität	41
6.2.3. Speicherung und Publish	42
6.3. Was sich über die Qualität der Vorschläge sagen lässt	42
6.4. Überleitung zu den Ergebnissen	43
7. Ergebnisse	44
7.1. Gesamtstand der Anforderungen	44
7.2. Was der Beispiel-Lauf über die Vorschläge sagt	46
7.3. Expertenbewertung der Tickettexte	47
7.4. Zustände eines Vorschlags	48
7.5. Reichweite des Ergebnisses	50
8. Diskussion	51
8.1. Was der Beispiel-Lauf zeigt	51
8.2. Warum die Kontrolllücken in genau diesem Lauf zählen	52
8.3. Was die Expertenbewertung zeigt – und was nicht	52
8.4. Grenzen, die aus diesem Material folgen	53
8.5. Diskussionsfazit	53
9. Fazit und Ausblick	54
9.1. Beantwortung der Forschungsfragen	54
9.1.1. Erforderliche Informationen eines Tickets	54

9.1.2. Kontrollierte Integration eines KI-Assistenten	55
9.2. Wissenschaftlicher und praktischer Beitrag	56
9.3. Priorisierter Ausblick	56
9.4. Schlussfolgerung	57
Quellen	59
A. Illustrierter Ticketverlauf	64
B. Dokumentierter Prüfstand	67
C. Ausgabeschema des Prototyps	68
D. Expertenbewertung	69

Abbildungsverzeichnis

4.1. Systemkontext und Komponenten des Dev-Assist-Prototyps mit Vertrauensgrenzen. Der Express-Dienst orchestriert GitLab-Zugriffe, KI-Analyse und lokale Kontextpersistenz. Rote Datenflüsse überschreiten eine Plattform-, Prozess- oder Anbietergrenze. Eigene Darstellung auf Grundlage des untersuchten Quellcodes, Stand 31.07.2026.	22
4.2. Sequenz der Analyse- und Publish-Verarbeitung über den GitLab-Webhook. GitLab erhält sofort eine Bestätigung (HTTP 202); die eigentliche Arbeit läuft danach ohne weitere Rückmeldung an den Absender. Rote Hinweise kennzeichnen Fehlerpfade und dass sichtbare Vorschau, gespeicherter Kontext und Freigabe nicht an dieselbe Fassung gebunden sind. Eigene Darstellung auf Grundlage des untersuchten Quellcodes, Stand 31.07.2026.	25
5.1. Anlegen eines unvollständigen Dev-Assist-Issues in GitLab. Die erste inhaltliche Zeile enthält @dev-assist. Der Screenshot stammt aus einem dokumentierten Lauf im untersuchten Projekt.	29
5.2. Klärungskommentar von Dev-Assist und anschließende Antworten im selben Issue. Der Screenshot stammt aus dem dokumentierten Lauf zu Abbildung 5.1.	30
5.3. Strukturierter Vorschlagskommentar nach der Klärung. Der Screenshot stammt aus dem dokumentierten Lauf zu Abbildung 5.1.	31
5.4. Dasselbe Issue nach @dev-assist publish. Titel und Beschreibung enthalten den gespeicherten Entwurf. Der Screenshot stammt aus dem dokumentierten Lauf zu Abbildung 5.1.	32
7.1. Soll-Zustandsmodell eines Dev-Assist-Vorschlags und Abweichung der aktuellen Implementierung. Die Freigabe muss an dieselbe Fassung gebunden sein, die vollständig in GitLab sichtbar war. Der rot gestrichelte Übergang zeigt die fehlende Prüfung von Status, Fassung und Freigabe. Eigene Darstellung auf Grundlage des untersuchten Quellcodes, Stand 31.07.2026.	49

Tabellenverzeichnis

2.1. Einordnung aktueller empirischer Arbeiten: Beitrag und Grenze	10
3.1. Informationsgruppen des abgeleiteten Qualitätsmodells	14
3.2. Herkunft und Nachverfolgbarkeit der Anforderungsgruppen	19
4.1. Architekturbereiche, Verantwortlichkeiten und zentrale Module	23
4.2. Fehlerklassen und vorgesehene Reaktionen im Zielablauf	26
5.1. Leistungsstand der Kernfunktionen im untersuchten Prototyp	33
5.2. Projektbereiche und umgesetzte Verantwortlichkeiten	34
6.1. Ergebnisse des dokumentierten Build- und Testlaufs	39
6.2. Untersuchte projektinterne Artefaktgruppen	40
7.1. Konsolidierter Anforderungs- und Nachweisstand des untersuchten Pro- totyps (eigene Darstellung)	45
7.2. Median der Expertenbewertung je Aussage und Ticket (n=6; Skala 1–5; eigene Erhebung, 18.08.2026)	48
9.1. Priorisierte Roadmap vom Machbarkeitsnachweis zur belastbaren Eva- luation	57
B.1. Kurzprotokoll des dokumentierten Build- und Testlaufs	67
C.1. Felder des kompakten Ausgabeschemas	68
D.1. Rohwerte der Expertenbewertung (1–5; eigene Erhebung, 18.08.2026)	70

Abkürzungsverzeichnis

API	Application Programming Interface
CLI	Command Line Interface
FA	funktionale Anforderung
HMAC	Hash-based Message Authentication Code
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JSONL	JSON Lines
LLM	Large Language Model
NFA	nicht-funktionale Anforderung
NIST	National Institute of Standards and Technology
OCR	Optical Character Recognition
OWASP	Open Worldwide Application Security Project
REST	Representational State Transfer
SRS	Software Requirements Specification
TTL	Time to Live

1. Einleitung

Software-Tickets sind in Entwicklungsteams mehr als Aufgabenlisten. Sie halten fest, was gebaut oder repariert werden soll, und dienen zugleich als Gesprächsraum zwischen meldenden Personen, Product Ownern und Entwicklung. In der Praxis sind diese Tickets häufig unvollständig.

In dieser Arbeit meint Ticket ein GitLab-Issue. Im betrachteten Team sind das User Stories und Fehlerberichte (Bugs). Jedes Ticket steht für sich. Es gibt keine Zusammenfassung in Epics, Features oder Themes und keine Richtlinie, wie ein Ticket anzulegen ist. Genau das macht die Aufbereitung relevant: Wer umsetzt, muss das einzelne Issue verstehen können. Es gibt kein übergeordnetes Epic und keine Vorlage, die fehlende Angaben erzwingt.

Forschung zu Bug Reports zeigt eine wiederkehrende Lücke: Meldende Personen liefern andere Angaben, als Entwicklungsteams für Umsetzung und Prüfung brauchen. Reproduktionsschritte, Testfälle und technischer Kontext fehlen oft oder tauchen erst später in Kommentaren auf (vgl. Bettenburg et al., 2008, S. 308 f.; Breu et al., 2010, S. 301 ff.). Auch kurze User Stories können formal lesbar sein und trotzdem fachlich lückenhaft oder praktisch nicht nutzbar bleiben (vgl. Lucassen et al., 2016, S. 383 f. und S. 386 f.).

Ein Ticket besitzt in GitLab bereits Titel, Beschreibung und Kommentarverlauf. Zusammengeführt werden muss dennoch der aktuelle Stand des Tickets. Die Beschreibung bleibt häufig beim ersten, unvollständigen Entwurf. Antworten, Entscheidungen und Akzeptanzkriterien entstehen später und stehen dann nur in den Kommentaren. Vor der Umsetzung muss dieser Verlauf gelesen werden, um den aktuellen Stand zu kennen. Zusammenführen bedeutet hier, aus Beschreibung und Kommentaren eine aktuelle Ticketbeschreibung zu erstellen. Der Stand liegt dann in der Beschreibung und muss nicht erst aus dem Kommentarverlauf rekonstruiert werden.

Die Problemstellung ist damit doppelt. Fachlich ist unklar, welche Angaben ein Ticket überhaupt enthalten muss, damit Umsetzung und Prüfung nachvollziehbar sind. Technisch ist unklar, wie Unterstützung durch ein Sprachmodell in genau diesem GitLab-Ablauf möglich ist, ohne das Ticket unkontrolliert zu verändern. Daraus folgen die beiden Leitfragen der Arbeit:

1. **Forschungsfrage 1:** Welche Informationen muss ein GitLab-Ticket enthalten, damit Entwicklerinnen und Entwickler es nachvollziehbar umsetzen und überprüfen können?
2. **Forschungsfrage 2:** Wie lässt sich ein KI-gestützter Assistent so in einen GitLab-basierten Entwicklungsablauf integrieren, dass Analyse, Rückfragen, Vorschlag und Freigabe kontrolliert ablaufen?

1.1. Ziel und Vorgehen

Ziel ist ein technischer Machbarkeitsnachweis, nicht der Ersatz menschlicher Anforderungsarbeit. Die erste Frage klärt, welche Angaben in einem Ticket stehen müssen, damit jemand damit umsetzen und das Ergebnis prüfen kann. Die zweite Frage klärt, unter welchen festen Regeln ein Sprachmodell in GitLab mitarbeiten darf.

Die Arbeit folgt dieser Reihenfolge. Kapitel 2 beschreibt die vorgefundene Umgebung und den Stand der Technik. Kapitel 3 entwickelt daraus die Lösungsidee. Kapitel 4 und 5 setzen sie als Prototyp um. Kapitel 6 bis 8 prüfen und diskutieren den gebauten Stand. Kapitel 9 beantwortet die Forschungsfragen.

Die Literatúrauswahl ist keine vollständige systematische Studie. Sie konzentriert sich auf Qualitätsmerkmale von User Stories und Bug Reports, auf automatisierte Qualitätshilfen und auf Risiken generativer Systeme. Ticketqualität meint hier, ob ein Ticket umsetzbar und prüfbar ist. Sie meint nicht die Qualität des fertigen Produkts und nicht die Leistung des Teams.

Eine Feldstudie oder ein Nachweis von Zeitgewinn ist nicht Bestandteil der Arbeit. Die Prüfung trennt den Anwendungsteil ohne Sprachmodell vom erzeugten Tickettext. Ob ein erzeugter Text fachlich richtig ist, können Übersetzung des Programms und automatisierte Tests nicht belegen. Eine kleine Expertenbewertung von drei Tickettexten ergänzt die technische Prüfung.

1.2. Aufbau und Abgrenzung

Kapitel 2 behandelt Umgebung und Stand der Technik, ohne bereits eine Lösung festzulegen. Kapitel 3 formuliert die Lösungsidee als Qualitätsmodell, Zielprozess und Anforderungen. Kapitel 4 und 5 beschreiben die Lösung als Architektur und Implementierung. Kapitel 6 bis 8 enthalten die kritische Reflexion. Kapitel 9 beantwortet die Forschungsfragen und priorisiert weiteren Bedarf. Ein illustrierter Ticketverlauf, der dokumentierte Prüfstand und das Ausgabeschema stehen im Anhang.

Epics, Features und Themes gehören nicht zum Untersuchungsgegenstand. Bewertet wird jeweils ein einzelnes GitLab-Issue. Langzeitdaten und eine statistische Auswertung vieler Tickets gehören nicht zur Untersuchung. Bildinhalte von Anhängen und Screenshots werden nicht gelesen oder ausgewertet. Der dokumentierte GitLab-Lauf in Kapitel 5 macht den Ablauf im Issue sichtbar. Kapitel 6 und 7 ergänzen eine kleine schriftliche Expertenbewertung (n=6) an drei echten Beispieltickets. Das ist keine Feldstudie und kein Experiment mit Versuchsgruppen. Die Arbeit kann deshalb zeigen, dass der Ablauf in GitLab ausführbar ist, welche Kontrollen der gebaute Stand einhält oder verfehlt und wie sechs Personen drei aufbereitete Texte beurteilen. Eine allgemeine Zeitersparnis oder Qualitätssteigerung im Feld lässt sich damit nicht nachweisen. Diese Grenze gilt für die Interpretation der Evaluation und der Forschungsfragen.

2. Umgebung und Stand der Technik

Dieses Kapitel beschreibt zuerst die vorgefundene Arbeitsumgebung und danach den Stand der Technik. Es legt noch keine Lösung fest. Die Ableitung der Lösungsidee folgt in Kapitel 3.

2.1. GitLab als vorgefundene Umgebung

2.1.1. Entwicklungsplattform

GitLab ist eine webbasierte Plattform für den Softwarelebenszyklus. Teams verwalten dort Quellcode, Merge Requests, CI/CD und Aufgaben in einem gemeinsamen Projekt. Der Issue Tracker ist der Ort, an dem Tickets erstellt, diskutiert und bearbeitet werden. Im betrachteten Team ist das die vorgefundene Umgebung: User Stories und Fehlerberichte entstehen in GitLab und werden dort gepflegt.

2.1.2. Issues und Kommentarverlauf

Ein Issue besteht aus Titel, Beschreibung, Metadaten und einem Kommentarverlauf. Die Erstbeschreibung ist nur der Ausgangspunkt. Rückfragen, Antworten und Entscheidungen entstehen später als Kommentare (in GitLab Notes). Der aktuelle Stand liegt dann häufig nicht mehr in der Beschreibung, sondern verteilt im Kommentarverlauf.

Über die GitLab-Schnittstellen lässt sich ein Issue lesen. Änderungen können auf zwei Wegen zurückgeschrieben werden: als neuer Kommentar oder durch Aktualisierung von Titel und Beschreibung (vgl. GitLab Documentation, o. J.c, o. S.; GitLab Documentation, o. J.d, o. S.).

2.1.3. Webhooks

Externe Dienste können über Webhooks angebunden werden. Ein Webhook ist eine automatische Benachrichtigung: Bei einem konfigurierten Ereignis sendet GitLab eine

HTTP-Anfrage mit den Ereignisdaten. Für Issue-Arbeit sind neue oder geänderte Issues sowie neue Kommentare die relevanten Auslöser (vgl. GitLab Documentation, o. J.a, Abschnitt "Webhook events"; GitLab Documentation, o. J.b, Abschnitt "Comment events"). Der Empfänger soll schnell antworten und längere Arbeit danach erledigen (vgl. GitLab Documentation, o. J.a, Abschnitt "Webhook receiver requirements"). Die Dokumentation verlangt außerdem, die Herkunft einer Lieferung zu prüfen (vgl. GitLab Documentation, o. J.a, Abschnitt "Signing tokens").

2.2. Requirements Engineering und Ticketqualität

Requirements Engineering umfasst das Erheben, Analysieren, Dokumentieren, Prüfen und Verwalten von Anforderungen über ihren Lebenszyklus. Die öffentlich zugängliche Katalogseite zu ISO/IEC/IEEE 29148 beschreibt diesen Prozessbereich (vgl. ISO/IEC/IEEE, 2018, o. S.). Der Volltext der Norm wird hier nicht vorausgesetzt. In der Fachliteratur werden mit diesem Standard Merkmale wie Eindeutigkeit, Vollständigkeit, Nachverfolgbarkeit und Prüfbarkeit verbunden (vgl. Lubos et al., 2024, S. 389–397; Paiva et al., 2026, Abschnitte 3 und 5). Für ein Ticket heißt das konkret: Ist klar, worum es geht? Steht genug da, um umzusetzen? Kann man das Ergebnis prüfen? Ist erkennbar, worauf sich eine Aussage stützt?

Ein GitLab-Ticket ist in dieser Arbeit ein einzelnes Issue, in der Regel eine User Story oder ein Fehlerbericht. Es ist keine vollständige Spezifikation und keine Gruppe von Stories unter einem Epic. Es ist ein kurzes Arbeitsdokument, das im Kommentarverlauf fortgeschrieben wird. Die genannten Merkmale helfen trotzdem, Aussagen und Lücken zu beurteilen. Welche Felder ein Ticket braucht, wird in Kapitel 3 aus den folgenden Befunden abgeleitet, nicht aus einer Prüfung gegen den Normvolltext.

2.2.1. Empirische Befunde zu User Stories und Bug Reports

In agilen Prozessen entstehen Anforderungen häufig als Tickets. In dieser Arbeit sind das User Stories und Fehlerberichte (Bugs). Das erleichtert die Zusammenarbeit in kurzen Schritten. Zugleich hängt viel davon ab, dass alle denselben Stand im Ticket vorfinden. Inayat et al. beschreiben agiles Requirements Engineering deshalb als stark kommunikations- und abstimmungsorientiert (vgl. Inayat et al., 2015, S. 915 ff.). Lucas-

sen et al. unterscheiden bei User Stories drei Ebenen: die Form eines Textes, seine Bedeutung und seine praktische Nutzbarkeit (vgl. Lucassen et al., 2016, S. 386 f.). Ein Text kann lesbar sein und trotzdem die falsche Bedeutung vermitteln oder für die Umsetzung unbrauchbar bleiben. Diese Dreiteilung bleibt später wichtig: Eine sauber gegliederte Ausgabe ist noch keine gute Anforderung.

Ein Fehlerbericht (Bug) beschreibt ein Verhalten, das so nicht erwartet wurde. Die Forschung zu Bug Reports zeigt, welche Angaben in solchen Tickets häufig fehlen. Bettenburg et al. zeigen, dass Entwicklungsteams Reproduktionsschritte, Stack Traces und Testfälle besonders hilfreich finden, meldende Personen diese Angaben aber nicht immer bereitstellen können (vgl. Bettenburg et al., 2008, S. 308 f.). Breu et al. untersuchten Fragen in 600 Bug Reports und identifizierten Nachfragen zu Reproduktion, Version, Betriebssystem, Beispielen, Ausgaben und Screenshots (vgl. Breu et al., 2010, S. 301 und S. 303). Chaparro et al. heben beobachtetes Verhalten, erwartetes Verhalten und Reproduktionsschritte als zentrale, zugleich häufig fehlende Bestandteile hervor (vgl. Chaparro et al., 2017, S. 396 f.).

Diese Befunde gelten nicht unverändert für beide hier betrachteten Ticketarten, User Story und Fehlerbericht. Die Literatur nennt aber denselben Kern an Angaben: einen eindeutigen Gegenstand, Ziel und Kontext, erwartetes oder gewünschtes Verhalten, eine Abgrenzung, überprüfbare Kriterien und sichtbare offene Punkte. Beim Fehlerbericht kommen Ist-Verhalten, Soll-Verhalten und Reproduktion hinzu. Bei der User Story stehen Nutzen, Umfang und Akzeptanzbedingungen im Vordergrund. Hinweise aus Logs, Fehlermeldungen oder dem Quellcode können ein Ticket stützen. Sie ersetzen aber keine fehlende fachliche Entscheidung.

Qualität bedeutet in dieser Literatur nicht möglichst viel Text. Aussage, Beleg und Unsicherheit sind zu unterscheiden. Eine Aussage ist, was das Ticket behauptet, etwa dass der Login auf Mobilgeräten fehlschlägt. Ein Beleg ist, worauf sich das stützt, etwa eine Fehlermeldung im Kommentar. Unsicherheit ist, was daraus noch nicht folgt, etwa ob alle Geräte betroffen sind. Diese drei Dinge dürfen nicht in einem Satz vermischt werden, der wie eine gesicherte Anforderung wirkt. Kapitel 3 überführt diesen Kern in ein Qualitätsmodell.

2.3. Large Language Models und agentenbasierte Assistenz

Große Sprachmodelle (Large Language Models) werden auf großen Textmengen vortrainiert und können Aufgaben anhand natürlichsprachlicher Anweisungen und Beispiele bearbeiten. Brown et al. zeigen mit GPT-3, dass Aufgaben ohne neues Training gelöst werden können, wenn die Aufgabe im Eingabetext beschrieben wird (vgl. Brown et al., 2020, S. 1877 und S. 1879). Die Gestaltung dieses Eingabetextes wird als Prompt Engineering bezeichnet (vgl. Zhao et al., 2023, S. 45 f.). Für Ticketarbeit ist das relevant, weil Ticketart, Sprache und Informationslage stark variieren und sich nicht vollständig in festen Einzelregeln abbilden lassen.

Ein Modell kann vorhandene Aussagen gruppieren, fehlende Kategorien markieren, Rückfragen formulieren und einen strukturierten Entwurf erzeugen. Das bedeutet nicht, dass es das System verstanden hat. Es kennt weder automatisch die fachliche Absicht noch den vollständigen Projektkontext. In der Literatur bleibt die Rolle Assistenz: Das Modell liefert einen Entwurf, keine gesicherte fachliche Entscheidung.

Solche Modelle können außerdem Werkzeuge aufrufen und Daten aus der Umgebung einbeziehen (vgl. Xi et al., 2023, S. 1; Yao et al., 2023, S. 1 f.; Schick et al., 2023, S. 68539 f.). Ein Modell wird erst dann Teil eines Entwicklungsablaufs, wenn Datenquellen, Schnittstellen und Rechte begrenzt sind. Welche Begrenzung für diese Arbeit gilt, legt Kapitel 3 fest.

2.4. Automatisierte Unterstützung und aktuelle LLM-Forschung

2.4.1. Regelbasierte und hybride Werkzeuge

Ticketqualität lässt sich teilweise automatisiert unterstützen. CUEZILLA bewertet Bug Reports und gibt Hinweise auf ergänzende Informationen (vgl. Bettenburg et al., 2008, S. 308 f.). AQUA macht formale und sprachliche Defekte in User Stories sichtbar (vgl. Lucassen et al., 2016, S. 383 f. und S. 386 f.). DeMIBuD erkennt fehlendes erwartetes Verhalten und fehlende Reproduktionsschritte (vgl. Chaparro et al., 2017, S. 397 und S. 401 ff.). BEE klassifiziert Bug Reports, markiert Observed Behavior, Expected Behavior und Steps to Reproduce und meldet fehlende Elemente (vgl. Song/Chaparro, 2020, S. 1551 f.). ChatBR kombiniert einen Klassifikator mit einem Sprachmodell, das als feh-

lend erkannte Bestandteile ergänzt. Über sechs Projekte berichten Bo et al. eine hohe strukturelle Vollständigkeit und eine mittlere semantische Ähnlichkeit der Ergänzungen von 84,10 Prozent (vgl. Bo et al., 2024, S. 1472–1483). Dass ein Feld gefüllt ist, heißt deshalb nicht, dass die Aussage im konkreten Projekt stimmt.

Diese Werkzeuge sind Forschungsprototypen für jeweils eine Artefaktart und eine Prüfungsaufgabe, etwa Bug Reports oder User Stories. Sie liegen nicht als GitLab-Dienste vor, die auf neue Kommentare reagieren, und erzeugen keine Ticketbeschreibung, die ein Mensch freigeben muss. Die Werkzeuge selbst sind nicht Gegenstand dieser Arbeit. Relevant bleiben die Qualitätskategorien und die Trennung von Lücken erkennen, Text erzeugen und Freigabe.

2.4.2. Empirische Arbeiten zu LLMs im Requirements Engineering

Die jüngere Forschung untersucht Sprachmodelle für konkrete RE-Aufgaben: Texte erzeugen oder überarbeiten, Qualität prüfen und verteilte Quellen verknüpfen. Die Studien liefern erste Belege. Ein sprachlich überzeugender Text und ein kontrollierter Arbeitsprozess sind dabei zwei verschiedene Dinge.

Krishna et al. vergleichen GPT-4 und CodeLlama bei Erstellung, Prüfung und Korrektur vollständiger Anforderungsspezifikationen. Potenzial für strukturierte Entwürfe und Zeitersparnis zeigte sich, fiel aber nach Modell und Aufgabe unterschiedlich aus (vgl. Krishna et al., 2024, S. 475–483). Die Studie arbeitet mit langen Spezifikationsdokumenten, nicht mit kurzen GitLab-Tickets, die durch Kommentare ergänzt werden. Ein Nachweis, dass ein Assistent in GitLab Zeit spart, folgt daraus nicht.

Lubos et al. setzen ein LLM als Prüfinstanz für Qualitätsmerkmale nach ISO/IEC/IEEE 29148 ein. Die Übereinstimmung zwischen Menschen und Modell war teilweise schwach. Erklärungen und überarbeitete Formulierungen wurden dennoch häufig als plausibel eingeschätzt (vgl. Lubos et al., 2024, S. 389–397). Das stützt einen assistierenden Einsatz und warnt davor, ein Modellurteil mit fachlichem Konsens gleichzusetzen.

Paiva et al. erzeugen aus Issue-Titeln von fünf Open-Source-Projekten Anforderungssätze und bewerten, ob sie eindeutig, prüfbar und auf jeweils einen Sachverhalt begrenzt sind. Die Autoren beschränken ihre Aussage ausdrücklich auf sprachlich wohl-

geformte Sätze bei sehr wenig Eingangskontext. Ob die Sätze zum konkreten Projekt passen und mit den Beteiligten abgestimmt sind, wird nicht nachgewiesen (vgl. Paiva et al., 2026, Abschnitte 3 und 5). Die Studien betrachten Beschreibung, Kommentare und weiteren Dokumentkontext oft getrennt. Ob zusätzlicher Repository-Kontext bessere Vorschläge erzeugt, prüfen sie nicht für GitLab-Issues.

K C et al. erzeugen zu 476 Feature Requests konkrete Rückfragen. 164 von 167 manuell bewerteten Fragen galten als korrekt. Erfahrene Entwicklerinnen und Entwickler bestätigten, dass Nachfragen bei mangelhaften Requests typischerweise der erste Schritt sind (vgl. K C et al., 2025, S. 104–116). Unklare Bedeutung blieb schwieriger als lokale Sprachfehler. Der Befund stützt Rückfragen, nicht die automatische Freigabe.

Weitere industrielle Studien bestätigen dieselbe Grenze mit unterschiedlichen Aufgaben. Projektnahe Beispiele verbessern die Erkennung von Mehrdeutigkeit, bleiben aber von Datensatz und Domäne abhängig (vgl. Bashir et al., 2025, S. 620–631). In einer medizinischen Spezifikation lagen die Werte für Unvollständigkeit, Widersprüche und Mehrdeutigkeit deutlich unter einer Schwelle, ab der man automatisieren könnte (vgl. Mahbub et al., 2024, S. 171972–171992). Zusätzliche Dokumente helfen, garantieren aber keine korrekte Antwort (vgl. Ibtasham et al., 2025, S. 277–292; Tizard et al., 2023, S. 2381 f. und S. 2389 ff.). Frattini et al. trennen deshalb drei Ebenen: die Eigenschaft einer Anforderung, ihre Bewertung und die Wirkung in der Nutzung (vgl. Frattini et al., 2023, S. 507–520). Ein sprachlich klarer Vorschlag ist zuerst ein formales Artefakt. Ob er Umsetzung oder Abstimmung verbessert, braucht eigene Nutzungsdaten.

Tabelle 2.1: Einordnung aktueller empirischer Arbeiten: Beitrag und Grenze

Arbeit	Untersuchter Beitrag	Grenze
Krishna et al. (2024)	Generierung und Prüfung vollständiger Anforderungsspezifikationen mit zwei Sprachmodellen	Lange Spezifikationen, keine GitLab-Tickets mit Kommentarverlauf
Lubos et al. (2024)	LLM-basierte Qualitätsprüfung nach ISO/IEC/IEEE 29148 mit menschlicher Bewertung	Die Modellerklärung unterstützt die Prüfung, ersetzt aber keinen fachlichen Konsens
K C et al. (2025)	Erkennung von Lücken und Generierung konkreter Rückfragen für Feature Requests	Dialog unterstützt die Verfeinerung. Semantische Lücken bleiben menschlich zu prüfen
Bo et al. (2024)	Hybride Erkennung und generative Ergänzung fehlender Bug-Report-Bestandteile	Strukturelle Vollständigkeit ist nicht mit faktischer Richtigkeit gleichzusetzen
Tizard et al. (2023)	Verknüpfung von Forum, Issue Tracker und Dokumentation	Kontextquellen gemeinsam betrachten und die Herkunft erhalten
Paiva et al. (2026)	Anforderungsgenerierung aus Issue-Titeln mit mehreren Prompts und Modellen	Sprachqualität bei wenig Kontext belegt nicht, dass der Satz zum Projekt passt
Ibtasham et al. (2025)	Industrielle kontextgestützte Beantwortung von Requirements-Fragen	Der Abruf zusätzlichen Kontexts unterstützt, garantiert aber keine korrekte Antwort
Frattini et al. (2023)	Theorie und Forschungsrahmen für Requirements Quality	Formale Textqualität von der Wirkung im Entwicklungsprozess trennen

Tabelle 2.1 macht den Unterschied der Untersuchungsgegenstände sichtbar. Die zitierten Arbeiten prüfen Generierung, Qualitätsurteil, Rückfragen oder Kontextabruf jeweils für sich. Keine von ihnen untersucht denselben GitLab-Ablauf aus Ereignis, Rückfrage, sichtbarer Vorschau und Freigabe.

2.5. Risiken generativer Assistenz

Die folgenden Risiken stammen aus der Fachliteratur. Sie betreffen generative Assistenz allgemein, noch nicht eine konkrete Architektur.

Das zentrale inhaltliche Risiko sind falsche oder unbelegte Modellinhalte. Huang et al. beschreiben Halluzinationen als plausibel wirkende, aber nicht faktische Aussagen (vgl. Huang et al., 2023, S. 1 f.). NIST verwendet dafür auch den Begriff Confabulation (vgl. National Institute of Standards and Technology, 2024, S. 4 und S. 6). Eine Prüfung der Datenform kann falsche Feldtypen abweisen, etwa eine Zahl statt einer Liste. Ob ein syntaktisch gültiger Satz wahr ist, kann sie nicht feststellen.

Ein zweites Risiko ist Prompt Injection: Tickettexte oder eingebundene Repository-Inhalte können versteckte Anweisungen enthalten, die das Modell umleiten sollen (vgl. Liu et al., 2023, S. 1; OWASP, 2025a, Abschnitt "LLM01:2025 Prompt Injection"). Diese Inhalte sind in der Literatur als Daten zu behandeln, nicht als Befehle. OWASP bezeichnet zu weitreichende Rechte eines Modells als übermäßige Handlungsfähigkeit (vgl. OWASP, 2025b, Abschnitt "LLM06:2025 Excessive Agency").

Ein drittes Risiko entsteht an der Integrationsgrenze: gefälschte Ereignisse, wiederholte Lieferungen und Endlosschleifen, wenn ein Dienst auf eigene Nachrichten reagiert. Die Literatur trennt diese Fälle von der eigentlichen Textgenerierung.

2.6. Zwischenfazit

Die vorgefundene Umgebung ist GitLab. Tickets entstehen dort als User Stories oder Fehlerberichte. Der aktuelle Stand liegt häufig verteilt in Beschreibung und Kommentaren.

Zum Stand der Technik: Ein umsetzbares Ticket braucht nach der User-Story- und Bug-Report-Forschung mehrere Angaben, nicht nur einen lesbaren Text. Sprachmodelle können strukturieren und nachfragen, ersetzen aber keine fachliche Entscheidung. Bestehende Werkzeuge prüfen einzelne Artefaktarten; sie sind keine GitLab-Dienste mit Rückfrage, Vorschau und Freigabe. Halluzination, Prompt Injection und zu weitreichende Modellrechte sind bekannte Risiken.

Diese Befunde begründen die Lösungsidee in Kapitel 3. Sie legen sie noch nicht fest.

3. Lösungsidee und Anforderungen

Dieses Kapitel entwickelt die Lösungsidee. Es geht vom heutigen GitLab-Alltag aus, leitet das Qualitätsmodell aus Kapitel 2 ab und beschreibt den gewünschten Ablauf. Erst danach folgen die nummerierten Anforderungen. Die Herkunftstabelle kommt zum Schluss, wenn die Nummern bekannt sind.

3.1. Ausgangsprozess

Im Ausgangsprozess erstellt eine Person ein Issue, in der Regel als User Story oder Fehlerbericht. Jedes Ticket steht allein; eine Vorlage oder Richtlinie zur Erstellung gibt es nicht. Fehlende Informationen werden in Kommentaren nachgefragt. Antworten, Screenshots und Entscheidungen ergänzen den Kontext schrittweise, bleiben aber häufig nur im Verlauf. Vor der Umsetzung muss eine bearbeitende Person daraus den aktuellen Stand in der Beschreibung herstellen, ohne den Kommentarverlauf wiederholt auswerten zu müssen. Die Arbeit geht von der Annahme aus, dass diese Zersplitterung Wartezeiten, wiederholte Rückfragen und schwer nachvollziehbare Übergaben begünstigt. Eine Prozessmessung dazu ist nicht Bestandteil der Untersuchung.

3.2. Qualitätsmodell für Ticketvorschläge

Aus den Befunden in Kapitel 2 folgt ein gemeinsames Modell für User Stories und Fehlerberichte. Es ist kein starres Formular. Ein Ticket gilt als umsetzbar, wenn vier Punkte erfüllt sind: Der Gegenstand ist verstanden, der Umfang ist abgegrenzt, das Ergebnis lässt sich gegen beobachtbare Kriterien prüfen, und verbleibende Unsicherheit ist erkennbar.

Ziel und Gegenstand sagen, was geändert werden soll und wozu, etwa "Login auf Mobilgeräten". **Kontext und Scope** nennen Voraussetzungen, Abhängigkeiten und bewusste Nicht-Ziele, etwa "nur iOS, nicht Android". **Verhalten und Kriterien** verbinden das erwartete Ergebnis mit prüfbaren Bedingungen. Beim Fehlerbericht gehören Ist-Verhalten, Soll-Verhalten und Reproduktion dazu. **Evidenz und Hinweise** trennen be-

legte Angaben wie eine Fehlermeldung von fachlichen Entscheidungen. **Unsicherheit** hält Fragen, Widersprüche und Annahmen sichtbar, statt sie still zu schließen.

Diese Merkmale lassen sich am Text prüfen (vgl. Lucassen et al., 2016, S. 386 f.; Lubos et al., 2024, S. 389–397; Paiva et al., 2026, Abschnitte 3 und 5; Tizard et al., 2023, S. 2381 f.; Bettenburg et al., 2008, S. 308 f.; Chaparro et al., 2017, S. 396 f.). Ein ausgefülltes Feld beweist für sich noch keine Qualität. Wesentliche Lücken sollen zu konkreten Rückfragen führen. Sind Ziel, Umfang und Kriterien hinreichend geklärt, kann ein Vorschlag entstehen. Offene Punkte bleiben darin sichtbar. Ein kurzer, belegter Entwurf kann geeigneter sein als ein ausführlicher Text mit Annahmen.

Tabelle 3.1 fasst die fünf Informationsgruppen zusammen.

Tabelle 3.1: Informationsgruppen des abgeleiteten Qualitätsmodells

Informationsgruppe	Inhalt und Zweck
Ziel und Gegenstand	Eindeutiger Titel, Problem oder gewünschter Nutzen sowie betroffene Rollen oder Systeme
Kontext und Scope	Relevante Voraussetzungen, Randbedingungen, Abhängigkeiten und bewusste Nicht-Ziele
Verhalten und Kriterien	Erwartetes Ergebnis, bei Fehlern Ist-/Soll-Verhalten und Reproduktion, außerdem prüfbare Akzeptanzkriterien
Evidenz und Hinweise	Belegte Logs, Fehlermeldungen, Plattform- und Repository-Informationen ohne erfundene Implementierungsdetails
Unsicherheit	Offene Fragen, Widersprüche, Risiken, Annahmen und erforderliche Validierungsschritte

Das fachliche Zielmodell sieht neben Titel und Beschreibung auch Herkunft der Angaben, Umfang, Akzeptanzkriterien, technische Hinweise, offene Fragen, Risiken und Prüfschritte vor. Es ist absichtlich breiter als ein Minimalformat. Ein formal vollständiges Formular darf fehlende Informationen nicht verdecken. Kann ein Bereich nicht belegt gefüllt werden, muss er als offen gekennzeichnet werden. Kapitel 4 und 5 beschreiben ein kompakteres Ausgabeschema mit sechs Feldern. Ob diese Verkürzung reicht, bewertet erst die Evaluation.

Für die Rollen ergeben sich unterschiedliche Nutzungskriterien. Rückfragen an mel-

dende Personen sollen fachliches Ziel, beobachtbares Verhalten und Erfolgskriterien betreffen, nicht interne Dateipfade. Product Owner benötigen eine klare Abgrenzung und sichtbare Annahmen. Entwicklung und Qualitätssicherung brauchen einen Entwurf, aus dem Umsetzung und Prüfung getrennt hervorgehen. Für Betreiber sind Herkunftsprüfung, Konfiguration, Logging und ein sicherer Fehlerzustand entscheidend.

3.3. Zielprozess

Die Lösungsidee ist ein Assistent in GitLab, der die Zusammenarbeit unterstützt, ohne sie zu ersetzen. Er heißt Dev-Assist. Der Zielprozess umfasst fünf Schritte. Eine Person adressiert den Assistenten zu Beginn der ersten inhaltlichen Zeile mit `@dev-assist`. Die Anwendung prüft das Ereignis und sammelt Issue- sowie Kommentar-Kontext. Bei wesentlichen Lücken veröffentlicht sie gezielte Rückfragen. Andernfalls erstellt sie einen strukturierten Vorschlag. Der Vorschlag bleibt zunächst sichtbar und unverbindlich. Erst `@dev-assist publish` übernimmt den gespeicherten Stand in das Issue.

Aus den Risiken in Kapitel 2 folgen vier Leitprinzipien dieses Ablaufs. Erstens wird nur verarbeitet, wenn jemand `@dev-assist` setzt. Zweitens sammelt die Anwendung Ticketdaten und trennt sie von festen Systemanweisungen. Drittens akzeptiert sie nur eine definierte JSON-Antwort. Viertens ändert sie das Issue erst nach einer sichtbaren Vorschau und einem gesonderten Freigabekommando. Das Modell erzeugt den Entwurf. Die Anwendung entscheidet, ob GitLab verändert wird.

3.4. Stakeholder und Priorisierung

Die Beteiligten verfolgen unterschiedliche Interessen, die sich nicht immer vereinbaren lassen. Meldende Personen sollen mit wenig Aufwand ein Ticket anlegen können. Entwicklung und Qualitätssicherung brauchen genug Angaben, um Annahmen zu vermeiden und prüfen zu können. Product Owner betrachten Nutzen und Umfang. Betreiber brauchen sichere Zugriffe und eine nachvollziehbare Fehlerdiagnose. Der Assistent darf diesen Ausgleich nicht durch erfundene Inhalte herstellen. Fehlt eine Entscheidung, etwa zum Umfang, muss er nachfragen statt raten.

Muss-Anforderungen sichern den Kernablauf und die Sicherheitsgrenzen: Aktivierung,

Abruf des Ticketkontexts, strukturierte Analyse, Speicherung, expliziter Publish sowie die Begrenzung externer Eingänge und Modellantworten. Soll-Anforderungen wie erweiterte Screenshot-Verarbeitung, zusätzliche GitLab-Metadaten und ausführlichere Fehlermeldungen erhöhen den Nutzen. Dashboard, Produktmetriken und ein zusätzliches Rollenmodell bleiben Kann-Erweiterungen. Diese Priorisierung beschreibt das Soll. Sie behauptet nicht, dass der gebaute Stand sämtliche Muss-Anforderungen erfüllt. Innerhalb des Muss-Umfangs haben korrekte Änderungen und menschliche Kontrolle Vorrang vor Komfort.

3.5. Funktionale Anforderungen

Die funktionalen Anforderungen beschreiben den kontrollierten Kernablauf. Ihre Nummerierung wird in der Evaluation wieder aufgegriffen.

FA-1 Explizite Aktivierung. Dev-Assist reagiert nur, wenn die erste inhaltliche Zeile einer Issue-Beschreibung oder eines Kommentars mit `@dev-assist` beginnt. Führende Leerzeichen und einfache Markdown-Präfixe dürfen toleriert werden.

FA-2 Relevante Ereignisse. Issue- und Kommentar-Ereignisse müssen in ein gemeinsames internes Format mit Projekt, Issue, Text, Autor und Aktion überführt werden.

FA-3 Kommandotrennung. Eine normale Aktivierung startet die Analyse; `@dev-assist publish` startet ausschließlich die Übernahme eines zuvor gespeicherten Vorschlags.

FA-4 Abruf des Ticketkontexts. Soweit verfügbar, sind das aktuelle Issue, der Kommentarverlauf und optional begrenzter Repository-Kontext einzubeziehen. Webhook-Daten dienen als Fallback.

FA-5 Gemeinsame Analyse. Titel, Beschreibung, Kommentare, erkannte Antworten und Zusatzkontext sind gemeinsam auszuwerten. Gesicherte Aussagen, technische Hinweise und offene Punkte müssen unterscheidbar bleiben.

FA-6 Gezielte Rückfragen. Fehlen wesentliche fachliche Angaben, stellt das System konkrete, einzeln beantwortbare Fragen zu Ziel, Scope, erwartetem Verhalten oder Prüfkriterien. Bereits beantwortete Fragen sollen nicht wiederholt werden.

FA-7 Strukturierter Vorschlag. Bei ausreichendem Kontext erzeugt Dev-Assist einen

als Issue-Beschreibung nutzbaren Entwurf. Unbelegte Inhalte dürfen nicht als Tatsachen dargestellt werden.

FA-8 Maschinenlesbares Ausgabeschema. Die Modellantwort muss einem definierten JSON-Schema entsprechen und vor jeder weiteren Verarbeitung geparkt und strukturell validiert werden.

FA-9 Sichtbare Rückgabe. Rückfragen, Vorschläge und Analysefehler werden als gekennzeichnete GitLab-Kommentare ausgegeben.

FA-10 Persistenz. Der analysierte Stand wird unter einem reproduzierbaren, gegen Pfadmanipulation begrenzten Speicherpfad als Markdown und optionale Metadaten gespeichert.

FA-11 Kontrollierter Publish. Publish liest ausschließlich den gespeicherten Stand, aktualisiert zuerst Titel und Beschreibung und bereinigt erst nach erfolgreicher Änderung Dev-Assist-Kommentare. Ohne erfolgreiche Analyse findet keine Übernahme statt.

3.6. Nicht-funktionale Anforderungen

Die nicht-funktionalen Anforderungen begrenzen Betrieb und Qualität. **NFA-1** fordert getrennte Module für Transport, GitLab, Analyse, Formatierung, Persistenz und Publish. **NFA-2** verlangt lokale Testbarkeit ohne reales GitLab und ohne kostenpflichtigen Modellaufruf. **NFA-3** betrifft Fallbacks und einen sicheren Abbruch bei ungültigem Kontext oder ungültiger Modellantwort. **NFA-4** fordert nachvollziehbare Konsolenlogs ohne Secrets.

NFA-5 begrenzt KI-Risiken durch feste Anweisungen, Schema-Validierung, sichtbare Unsicherheit und menschliche Freigabe. **NFA-6** fordert, dass Webhooks standardmäßig als von GitLab stammend erkannt werden müssen und alte oder wiederholte Lieferungen nicht als neu gelten; unsicherer Betrieb darf nur ausdrücklich für lokale Entwicklung aktiviert werden. **NFA-7** verhindert, dass der Assistent auf eigene Kommentare reagiert und dieselbe Lieferung kurz hintereinander doppelt verarbeitet. **NFA-8** trennt eine schnelle Empfangsbestätigung an GitLab von der längeren Modellarbeit. **NFA-9** verlangt Konfiguration über Umgebungsvariablen und die Trennung von Secrets und

Quellcode. **NFA-10** erhält die Bedienung vollständig im GitLab-Workflow und verlangt keine zusätzliche Oberfläche.

Die Anforderungen werden für den Machbarkeitsnachweis als Muss-Anforderungen behandelt, soweit sie den Kernprozess und seine Sicherheitsgrenzen betreffen. Erweiterte Bildanalyse, zusätzliche GitLab-Metadaten und differenzierte Fehlerdiagnosen sind Soll-Anforderungen. Dashboard, quantitative Produktmetriken und ein erweitertes Rollenmodell sind mögliche spätere Funktionen.

3.7. Herkunft und Nachverfolgbarkeit

Die Anforderungen stammen aus vier Quellen. Erstens liefern die in Kapitel 2 ausgewerteten Arbeiten die Qualitätskategorien. Zweitens zeigt der manuelle Ist-Prozess, an welchen Stellen Kontext verteilt ist. Drittens beschreibt die projektinterne Zielsetzung Aktivierung, Analyse, Klärung und Publish. Viertens begrenzen GitLab-Dokumentation, Sicherheitsrisiken und der Technologie-Stack die technische Ausgestaltung.

Tabelle 3.2 ordnet diese Quellen den nun eingeführten Anforderungen zu. Die Zuordnung verhindert, dass das Soll nur aus dem vorhandenen Code rückwärts abgeleitet wird. Der Prototyp wird später gegen dieses Zielbild geprüft. Abweichungen bleiben sichtbar.

Tabelle 3.2: Herkunft und Nachverfolgbarkeit der Anforderungsgruppen

Quelle	Abgeleiteter Bedarf	Anforderungsbezug
Forschung zur Ticketqualität	Fehlende Angaben erkennen, Verhalten und Prüfbarkeit trennen	Qualitätsmodell, FA-5 bis FA-8
GitLab-Ist-Prozess	Beschreibung und Kommentare gemeinsam behandeln, Interaktion im Issue halten	FA-1 bis FA-4, FA-6, FA-9
KI- und Sicherheitsrisiken	Modellantwort begrenzen, Unsicherheit erhalten, menschlich freigeben	FA-8, FA-11, NFA-3, NFA-5 bis NFA-7
Betrieb und Evaluation	Module isolieren, Ereignisse nachvollziehen und externen Zugriff konfigurieren	NFA-1, NFA-2, NFA-4, NFA-8 bis NFA-10

Die Nummerierung macht die Herkunft nachvollziehbar. Kapitel 4 ordnet den Anforderungen Architekturgrenzen zu. Kapitel 5 beschreibt, was der Prototyp tatsächlich tut. Kapitel 6 prüft beides gegen das Soll. Ein späterer Test belegt nur die konkret geprüfte Eigenschaft. Er darf hier noch nicht als Nachweis gelten, dass eine Anforderung erfüllt oder verletzt ist.

3.8. Abgrenzung und Zwischenfazit

Dev-Assist implementiert keinen Produktivcode, erstellt keine Merge Requests und priorisiert keine Tickets. Er ersetzt weder fachliche Entscheidungen noch die Prüfung eines Vorschlags. Die Arbeit bewertet die Struktur und technische Verarbeitung von Ticketinformationen, nicht die Qualität des gesamten Projekts. Eine eigene Bilderkennung, die Text aus Screenshots ausliest, ist nicht Teil des Prototyps. Ob ein Vorschlag fachlich wahr ist, kann das System nicht garantieren.

Damit verbindet die Lösungsidee zwei Ebenen. Fachlich müssen Ziel, Kontext, Verhalten, Kriterien, Belege und offene Punkte sichtbar sein. Technisch muss geprüft werden, ob eine Meldung von GitLab stammt; Kontext und Modellantwort sind zu begrenzen.

Ergebnisse sind nachvollziehbar zu speichern. Änderungen hängen von einer menschlichen Freigabe ab. Kapitel 4 setzt diese Idee in eine Architektur um.

4. Konzeption der Systemarchitektur

Dieses Kapitel setzt den Zielprozess aus Kapitel 3 in eine Architektur um. Der sichtbare Ablauf bleibt derselbe: Analyse nach `@dev-assist`, Freigabe nach `@dev-assist publish`. Das Modell darf Inhalte strukturieren. Die Anwendung entscheidet, ob ein Ereignis verarbeitet wird, welcher Kontext das Modell erreicht und ob GitLab verändert wird.

Drei Entwurfsfragen strukturieren die Lösung. Wie wird der Assistent ausgelöst, ohne GitLab dauernd abzufragen? Wer darf das Issue tatsächlich schreiben? Was darf das Modell allein festlegen? Die Antworten ergeben einen modularen Webhook-Dienst.

4.1. Systemkontext und Verantwortlichkeiten

Abbildung 4.1 zeigt die beteiligten Systeme und die Übergänge zwischen ihnen. Vier Bereiche sind zu unterscheiden. GitLab verwaltet das verbindliche Issue und erzeugt Ereignisse. Der lokale Dev-Assist-Host enthält die Anwendung, den GitLab-Client und die gespeicherten Kontextdateien. OpenCode bildet eine gesonderte Ausführungsgrenze für den Analyseagenten. Ein konfigurierter Modellanbieter liegt außerhalb des lokalen Hosts.

Jeder rote Pfeil in der Abbildung überschreitet eine Vertrauensgrenze. Ein Webhook darf erst nach Prüfung der Herkunft fachlich verarbeitet werden. Eine Modellanfrage darf nur den vorbereiteten Kontext enthalten. Die Antwort gilt als Vorschlag, nicht als Tatsache. Eine Änderung in GitLab darf nur über die Anwendung erfolgen, nicht durch das Modell selbst.

Die Architektur verfolgt vier Ziele: klare Verantwortlichkeiten, Auslösung durch Ereignisse, begrenzte Rechte des Modells und Nachvollziehbarkeit. Getrennte Module erleichtern Tests und Änderungen (vgl. Bass et al., 2021, Kap. 1). Webhooks vermeiden, dass der Dienst GitLab dauernd abfragt (vgl. Hohpe/Woolf, 2003, Abschnitte “Messaging” und “Message Channel”). Vorschau und Publish begrenzen, was das Modell allein bewirken kann. Gespeicherte Kontextdateien machen den Zwischenstand prüfbar.

Zwei Alternativen wurden verworfen. Ein Dienst, der Issues regelmäßig abfragt, hätte

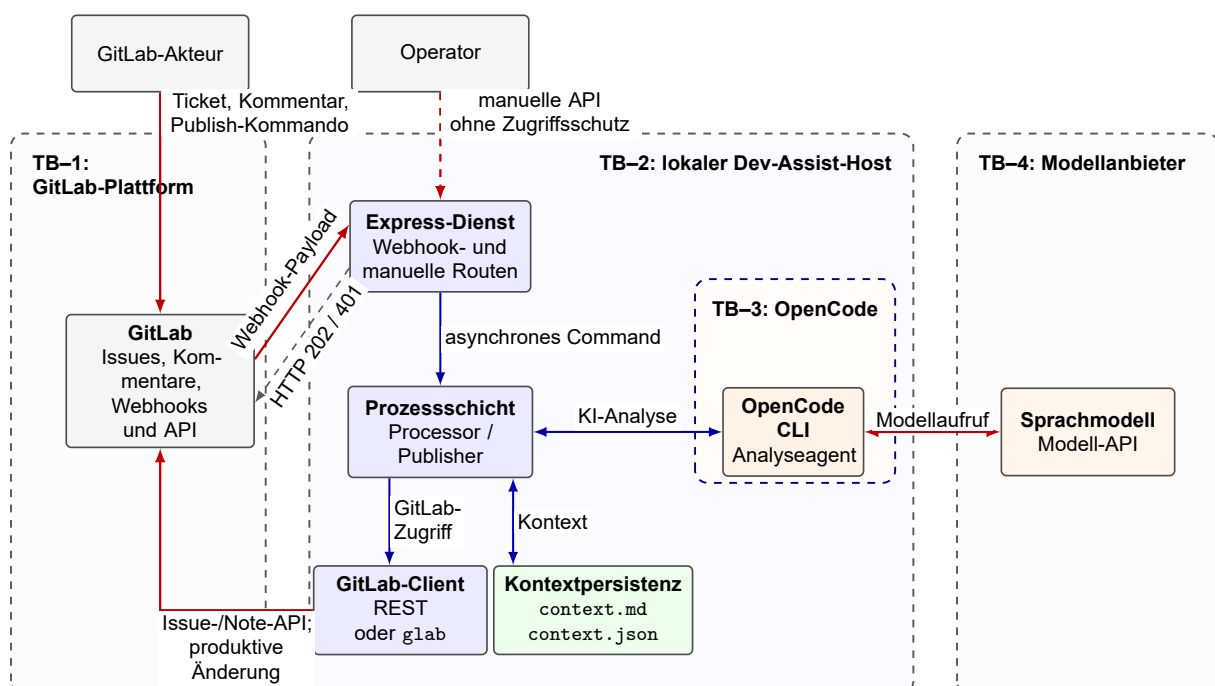


Abbildung 4.1.: Systemkontext und Komponenten des Dev-Assist-Prototyps mit Vertrauensgrenzen. Der Express-Dienst orchestriert GitLab-Zugriffe, KI-Analyse und lokale Kontextpersistenz. Rote Datenflüsse überschreiten eine Plattform-, Prozess- oder Anbietergrenze. Eigene Darstellung auf Grundlage des untersuchten Quellcodes, Stand 31.07.2026.

keine öffentliche Webhook-Grenze, würde aber Verzögerung und zusätzlichen Zustand erzeugen. Ein Agent mit direkten GitLab-Werkzeugen würde Modellentscheidung und Änderung des Issues vermischen. Für die Forschungsfrage ist die gewählte Trennung geeigneter: Auslösung, Analyse und Freigabe bleiben getrennt beobachtbar.

Tabelle 4.1 ordnet die Verantwortlichkeiten den später untersuchten Modulen zu. Die Pfade dienen der Nachverfolgbarkeit, nicht als Einstieg.

Tabelle 4.1: Architekturbereiche, Verantwortlichkeiten und zentrale Module

Bereich	Verantwortung	Zentrale Module
HTTP und Webhooks	Annahme, Authentifizierung, Parsing und schnelle Antwort	src/app.ts, src/routes/, src/services/gitlab/auth.ts
GitLab und Kontext	Issues und Notes lesen oder ändern, Bot-Filter, Kommentar-Bereinigung	src/services/gitlab/
Analyse und Ausgabe	Prompt, Mock/OpenCode, Schema, Rückfragen und Markdown	src/services/ai/, opencode.json
Prozess und Persistenz	Analyse orchestrieren, Kontextdateien verwalten und Publish ausführen	src/services/processing/, src/services/context/

4.2. Zentrale Entwurfsentscheidungen

Die erste Entscheidung betrifft die Auslösung. Der Assistent reagiert auf GitLab-Ereignisse, statt Issues regelmäßig abzufragen. Das passt zum Kommentarverlauf. Es erzeugt zugleich eine öffentliche HTTP-Grenze. Dort muss geprüft werden, ob die Meldung von GitLab stammt und ob sie nicht mehrfach verarbeitet wird.

Die zweite Entscheidung trennt Transportbestätigung und Analyse. GitLab erwartet eine schnelle erfolgreiche HTTP-Antwort (vgl. GitLab Documentation, o. J.a, Abschnitt "Webhook receiver requirements"). Der Prototyp antwortet mit Status 202: angenommen, aber fachlich noch nicht abgeschlossen. Die längere Modellarbeit läuft danach. Eine dauerhafte Warteschlange wäre im Betrieb robuster. Der Prototyp nutzt nur Hin-

tergrundauführung.

Die dritte Entscheidung trennt Analyse und Publish. Eine sofortige Übernahme würde eine fehlerhafte Modellantwort unmittelbar ins Issue schreiben. Deshalb speichert die Anwendung einen Zwischenstand. Publish darf nur diesen Stand übernehmen und keinen neuen Modelllauf auslösen. Der gespeicherte Stand soll genau dem zuvor sichtbaren Vorschlag entsprechen. Kapitel 5 zeigt, wie Analyse und Publish im Prototyp umgesetzt sind. Ob die Bindung gehalten wird, prüft Kapitel 6.

Die vierte Entscheidung begrenzt Fehler nach ihrer Wirkung. Fehlt Zusatzkontext, etwa die Repository-Zusammenfassung, darf ein gekennzeichnete Ersatztext verwendet werden. Eine ungültige Modellantwort erzeugt keinen speicherbaren Publish-Stand. Scheitert die Issue-Aktualisierung, müssen Diskussion und Ausgangsticket erhalten bleiben. Das Löschen der Dev-Assist-Kommentare darf im Soll erst erfolgen, wenn Titel und Beschreibung erfolgreich geschrieben wurden.

Diese Entscheidungen erzeugen Zielkonflikte. Eine frühe HTTP-Antwort vermeidet Timeouts, verlagert fachliche Fehler aber in Logs und Kommentare. Ein striktes Schema erleichtert Tests und kann fachliche Kategorien ausblenden. Zusätzlicher Repository-Kontext macht technische Hinweise plausibler und vergrößert zugleich die Angriffsfläche. Kontrolle des Ablaufs hat Vorrang vor einer möglichst autonomen Bedienung.

4.3. Analyse- und Publish-Pfad

Abbildung 4.2 zeigt die beiden Pfade, die eine Person auslöst. Zuerst die Analyse nach `@dev-assist`, danach die Freigabe nach `@dev-assist publish`. Das ist der Sollablauf. Rot markierte Stellen in der Abbildung gehören nicht zum Soll. Kapitel 5 zeigt die Umsetzung. Kapitel 6 prüft Abweichungen vom Soll.

Im Analysepfad lädt die Anwendung nach Möglichkeit das aktuelle Issue und die Kommentare. Scheitert dieser Abruf, bleibt der Text aus der GitLab-Meldung als begrenzter Ersatz. Optional ergänzt eine Repository-Zusammenfassung technische Hinweise. Feste Anweisungen und Ticketdaten werden getrennt an den Analyseagenten übergeben. Nur eine JSON-Antwort mit den sechs geforderten Feldern wird akzeptiert. Je nach

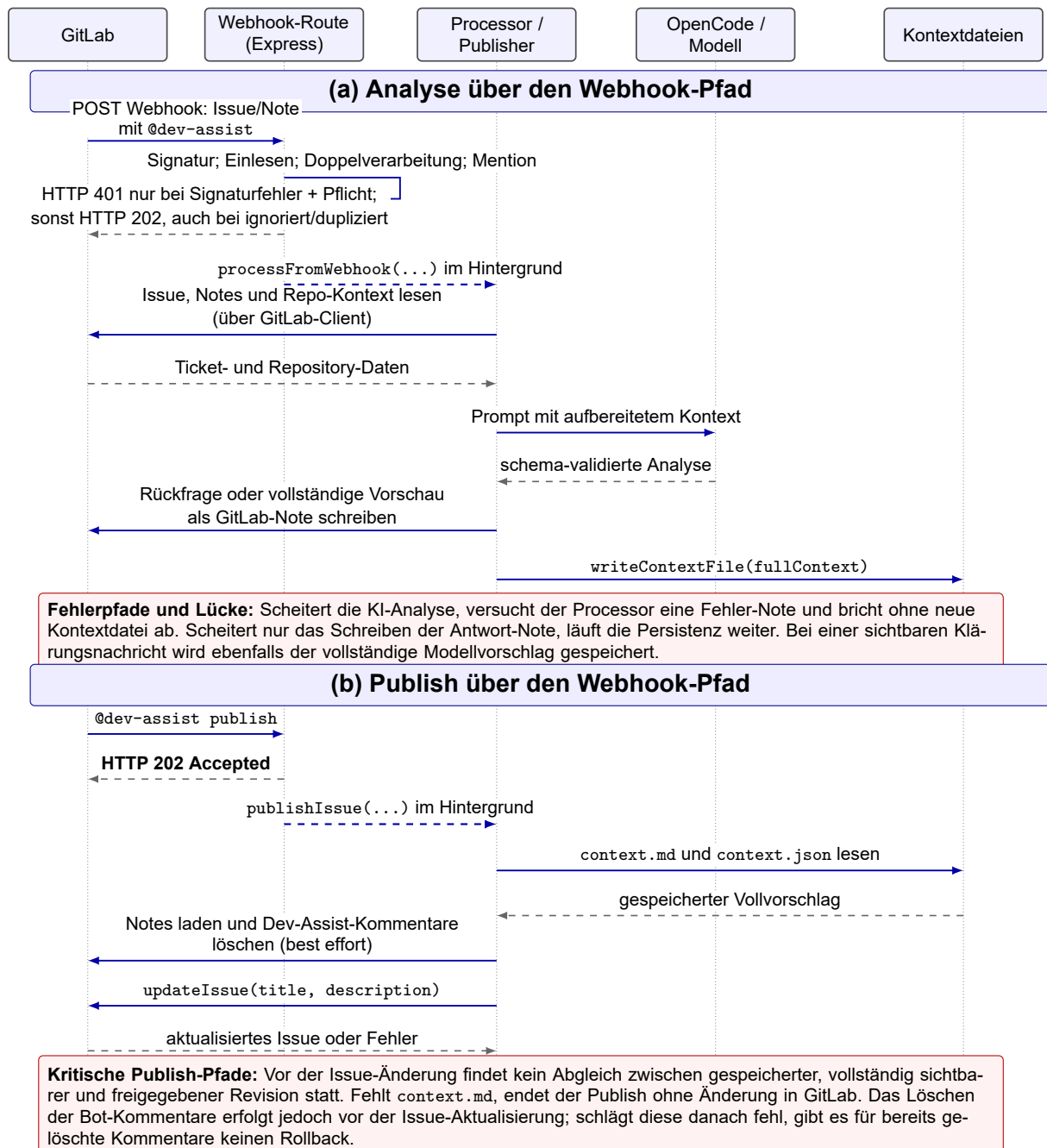


Abbildung 4.2.: Sequenz der Analyse- und Publish-Verarbeitung über den GitLab-Webhook. GitLab erhält sofort eine Bestätigung (HTTP 202); die eigentliche Arbeit läuft danach ohne weitere Rückmeldung an den Absender. Rote Hinweise kennzeichnen Fehlerpfade und dass sichtbare Vorschau, gespeicherter Kontext und Freigabe nicht an dieselbe Fassung gebunden sind. Eigene Darstellung auf Grundlage des untersuchten Quellcodes, Stand 31.07.2026.

offenen Fragen entsteht ein Klärungskommentar oder ein sichtbarer Entwurf. Parallel wird der Stand lokal gespeichert.

Kommentar und Kontextdatei sind zwei getrennte Wirkungen. Ein sichtbarer Klärungskommentar kann weniger enthalten als die gespeicherte Datei. Genau das ist die Lücke zwischen Vorschau und Freigabe. Publish soll deshalb nur den Stand übernehmen, der vollständig sichtbar war. Im Soll aktualisiert der Publisher zuerst das Issue und löscht Dev-Assist-Kommentare erst danach. Das Modell wird in diesem Pfad nicht erneut aufgerufen.

Tabelle 4.2 fasst die vorgesehene Reaktion auf typische Fehler zusammen.

Tabelle 4.2: Fehlerklassen und vorgesehene Reaktionen im Zielablauf

Fehlerklasse	Systemreaktion	Begründung
Ungültiger Webhook	Ablehnung vor fachlicher Verarbeitung	Externe Eingänge dürfen keine internen Schritte auslösen.
Fehlender Zusatzkontext	Warnung und begrenzter Fallback	Ein Vorschlag kann möglich bleiben, muss aber Kontextlücken erkennen lassen.
Ungültige Modellantwort	Kein normaler Kontext; Fehler wird sichtbar gemacht	Strukturell unkontrollierter Modelltext darf nicht in einen Publish gelangen.
Fehlgeschlagenes Issue-Update	Publish-Abbruch, Kommentare bleiben erhalten	Die bestehende Diskussion darf nicht vor der dauerhaften Änderung verloren gehen.
Fehler beim Löschen der Dev-Assist-Kommentare	Protokollierung nach erfolgreichem Update	Die Kernänderung ist bereits erfolgt; das Löschen bleibt nachgeordnet.

4.4. Was das Modell ausgeben darf

Der Prompt trennt feste Regeln von nicht vertrauenswürdigen Ticketdaten. Er fordert Ziel, Nutzen, Beschreibung, Akzeptanzkriterien, belegten technischen Kontext, Lösungsvorschlag und offene Fragen. Repository-Hinweise dürfen Technik stützen, aber keine fachlichen Anforderungen erfinden.

Das Ausgabeschema enthält genau sechs Felder: Titel, Beschreibung, Akzeptanzkriterien, technischen Kontext, Lösungsvorschlag und offene Fragen. Das Format ist leichter zu prüfen als freier Fließtext. Es bildet das breitere fachliche Zielmodell aus Kapitel 3 jedoch nicht vollständig ab. Quellenbasis, Risiken und Validierungsschritte haben keine eigenen Felder. Diese Differenz bleibt in der Evaluation sichtbar.

Bilder und Screenshots werden nicht heruntergeladen oder inhaltlich analysiert. Die Repository-Zusammenfassung bleibt auf Metadaten, Sprachen, einen Dateibaum-Ausschnitt und wenige Schlüsseldateien begrenzt.

4.5. Sicherheits- und Testbarkeitsziel

Die Architektur fordert, dass GitLab jede Lieferung mit einem geheimen Schlüssel signiert und die Anwendung diese Signatur sowie den Zeitstempel prüft. So zählen gefälschte oder alte Anfragen nicht (vgl. GitLab Documentation, o. J.a, Abschnitt "Signing tokens"). Die Signaturpflicht soll standardmäßig aktiv sein. Ein toleranter Modus ist nur für lokale Entwicklung vorgesehen. Manuelle Process- und Publish-Wege brauchen einen eigenen Zugangsschutz, weil sie dieselbe Logik ohne Webhook durchlaufen.

Zusätzlich sollen die Aktivierungsregel, das Ignorieren eigener Bot-Kommentare und das Unterdrücken kurzfristiger Doppelverarbeitung unbeabsichtigte Läufe verhindern. Projekt- und Issue-Kennungen müssen vor der Speicherung auf erlaubte Werte begrenzt werden, etwa positive Ganzzahlen. Ein Mock-Provider macht Parser, Schema und Formatter ohne Modellkosten prüfbar. Kapitel 5 beschreibt die Umsetzung. Kapitel 6 prüft sie gegen das Soll.

4.6. Zwischenfazit

Die Architektur beantwortet die drei Entwurfsfragen durch Webhooks, getrennte Freigabe und ein festes Ausgabeschema. Kontrolle entsteht außerhalb des Modells, in der Anwendung. Kapitel 5 zeigt, wie dieser Entwurf im TypeScript-Prototyp umgesetzt ist.

5. Implementierung

Dieses Kapitel beschreibt die gebaute Lösung. Dev-Assist besitzt keine eigene Oberfläche. Die Bedienung liegt vollständig in GitLab. Zuerst der sichtbare Ablauf, dann der Projektaufbau, dann die zentralen Stellen für Webhook, Analyse und Publish. Eine vollständige Datei-zu-Datei-Beschreibung ist nicht das Ziel. Ob die Umsetzung die Anforderungen aus Kapitel 3 einhält, prüft Kapitel 6.

5.1. Was eine Person im Issue sieht

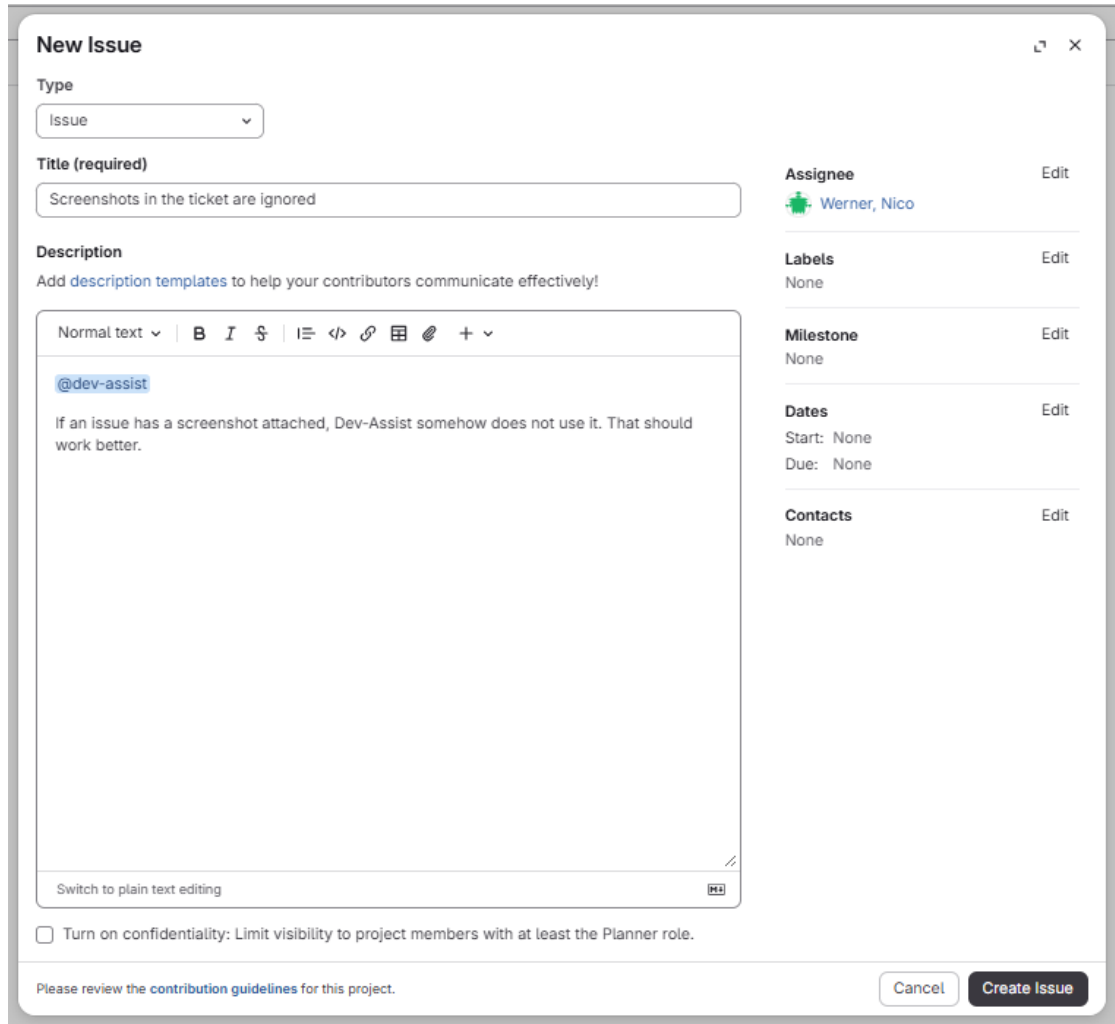
Die folgenden Screenshots stammen aus einem dokumentierten GitLab-Lauf im untersuchten Dev-Assist-Projekt. Das Beispielissue beschreibt, dass angehängte Screenshots bei der Analyse nicht gelesen werden. Es handelt sich um ein eigens angelegtes Demonstrationsissue, nicht um betriebliche Originaldaten und nicht um einen Wirksamkeitsnachweis.

Abbildung 5.1 zeigt den Einstieg. Titel und Beschreibung bleiben absichtlich unvollständig. Die erste inhaltliche Zeile enthält `@dev-assist`.

Abbildung 5.2 zeigt die Antwort im selben Issue. Der Dienst stellt zwei fachliche Rückfragen zu Umfang und sichtbarer Verwendung der Screenshots. Die meldende Person beantwortet sie in einem weiteren Kommentar, wiederum mit führendem `@dev-assist`.

Nach den Antworten erscheint ein strukturierter Vorschlag als Kommentar. Abbildung 5.3 zeigt diesen Entwurf mit Ziel, Akzeptanzkriterien, technischem Kontext und Lösungsvorschlag. Erst `@dev-assist publish` schreibt den gespeicherten Stand in Titel und Beschreibung. Abbildung 5.4 zeigt das Issue danach. Die Dev-Assist-Kommentare sind entfernt. Der Wortlaut des Verlaufs steht in Anhang A.

Tabelle 5.1 übersetzt denselben Ablauf in den umgesetzten Stand der Kernfunktionen.



New Issue

Type
Issue

Title (required)
Screenshots in the ticket are ignored

Description
Add [description templates](#) to help your contributors communicate effectively!

Normal text | **B** *I* U | **≡** `<>` [Link](#) [Image](#) [Video](#) [Audio](#) [Code](#) [More](#)

@dev-assist

If an issue has a screenshot attached, Dev-Assist somehow does not use it. That should work better.

Switch to plain text editing

☐ Turn on confidentiality: Limit visibility to project members with at least the Planner role.

Please review the [contribution guidelines](#) for this project.

Assignee: Werner, Nico Edit

Labels: None Edit

Milestone: None Edit

Dates: Start: None Due: None Edit

Contacts: None Edit

Cancel Create Issue

Abbildung 5.1.: Anlegen eines unvollständigen Dev-Assist-Issues in GitLab. Die erste inhaltliche Zeile enthält @dev-assist. Der Screenshot stammt aus einem dokumentierten Lauf im untersuchten Projekt.



dev-assist
@project_599_bot_40f728b5658b352de44dd525734c4cf9
just now

Maintainer
😊
↩
✎
⋮

Dev-Assist: More information needed

I looked at the issue, but there is not enough detail yet to create a solid, developer-ready ticket.

Important: I only ask about requirements, user needs, acceptance criteria, scope and success criteria. I do **not** need details about the current codebase, tech stack, specific components or existing implementation — the developer will handle those.

Please reply with answers to these questions (mention `@dev-assist` again with the details):

- Should screenshot support apply only to attachments on the main issue, or also to images added later in comments?
- Should Dev-Assist surface screenshot findings explicitly in the generated ticket text, or use them only internally to improve analysis quality?

Once I have the missing pieces I will post a full structured proposal and you can use `@dev-assist publish` to apply it.

(Current best guess for title: Use issue screenshots during Dev-Assist analysis)



Werner, Nico @nico.werner just now

Author
Maintainer
😊
↩
✎
⋮

`@dev-assist`

- Should screenshot support apply only to attachments on the main issue, or also to images added later in comments?
 - Screenshot support should apply to attachments and comments
- Should Dev-Assist surface screenshot findings explicitly in the generated ticket text, or use them only internally to improve analysis quality?
 - The screenshots should surface in the generated ticket

Abbildung 5.2.: Klärungskommentar von Dev-Assist und anschließende Antworten im selben Issue. Der Screenshot stammt aus dem dokumentierten Lauf zu Abbildung 5.1.



dev-assist @project_599_bot_40f728b5658b352de44dd525734c4cf9 1 minute ago

Maintainer

Dev-Assist: Structured Proposal

I have generated a structured proposal for this ticket/issue based on the details provided.

If you approve of these details, reply with `@dev-assist publish` to apply it. The conversation comments will then be cleaned up automatically.



Description

Dev-Assist should include screenshots from both the main issue and later comments when analyzing a GitLab ticket so that visual information is not ignored during ticket generation. This should improve the completeness and accuracy of the generated developer-ready ticket when key requirements, errors, UI states, or workflows are shown only in attached images.

The generated ticket should explicitly surface relevant findings from screenshots instead of using them only internally. Scope includes collecting screenshots attached to the issue description and comments, incorporating their content into the analysis workflow, and reflecting meaningful visual findings in the resulting structured ticket text when they add functional or contextual value.



Acceptance Criteria

- When an issue contains one or more screenshots in the main issue content, Dev-Assist uses them during analysis instead of ignoring them.
- When screenshots are added in issue comments, Dev-Assist also includes them in the analysis scope.
- The generated structured ticket explicitly reflects relevant information derived from screenshots when that information affects the described problem, requirements, scope, or acceptance criteria.
- If both text and screenshots are present, the output combines both sources into a coherent ticket rather than relying on text alone.
- If no screenshots are present, the existing text-only analysis flow continues to work without regression.
- Automated or manual validation demonstrates that issues with screenshot-only or screenshot-heavy context produce materially more complete ticket output than before.



Technical Context & Logs

- The current issue reports that Dev-Assist does not use screenshots attached to GitLab issues.
- Prior clarification confirms that screenshot support must apply to attachments on the main issue and to images added later in comments.
- Prior clarification confirms that screenshot findings should surface in the generated ticket text.
- The repository summary shows that GitLab issue processing is coordinated through `src/services/processing/*`, AI integration lives under `src/services/ai/*`, and GitLab access/parsing lives under `src/services/gitlab/*`.
- The repository summary states that Dev-Assist processes GitLab issue and note webhook content and uses the `apencode` CLI with the `dev-assist-analyzer` agent for real analysis.
- Assumption: screenshot handling may depend on whether the active AI/runtime path can receive image inputs or extracted image context in addition to text.



Proposed Solution

- Extend issue and comment context collection so screenshot attachments and embedded images from the main issue and subsequent comments are detected and included in the analysis input.
- Normalize the collected screenshot references and associated metadata into the issue context passed through the processing pipeline.
- Update the AI analysis flow so screenshot content is available alongside textual issue content, using direct image-capable input when supported or a fallback representation when necessary.
- Adjust the analysis instructions or formatting logic so relevant screenshot findings are explicitly reflected in the generated ticket description, acceptance criteria, or technical context when applicable.
- Add automated tests covering screenshots on the main issue, screenshots in comments, mixed text-plus-image issues, and the no-screenshot path to prevent regressions.

Abbildung 5.3.: Strukturierter Vorschlagskommentar nach der Klärung. Der Screenshot stammt aus dem dokumentierten Lauf zu Abbildung 5.1.

Use issue screenshots in Dev-Assist analysis

  Issue created 6 minutes ago by Werner, Nico

Description

Dev-Assist should include screenshots from both the main issue and later comments when analyzing a GitLab ticket so that visual information is not ignored during ticket generation. This should improve the completeness and accuracy of the generated developer-ready ticket when key requirements, errors, UI states, or workflows are shown only in attached images.

The generated ticket should explicitly surface relevant findings from screenshots instead of using them only internally. Scope includes collecting screenshots attached to the issue description and comments, incorporating their content into the analysis workflow, and reflecting meaningful visual findings in the resulting structured ticket text when they add functional or contextual value.

Acceptance Criteria

- When an issue contains one or more screenshots in the main issue content, Dev-Assist uses them during analysis instead of ignoring them.
- When screenshots are added in issue comments, Dev-Assist also includes them in the analysis scope.
- The generated structured ticket explicitly reflects relevant information derived from screenshots when that information affects the described problem, requirements, scope, or acceptance criteria.
- If both text and screenshots are present, the output combines both sources into a coherent ticket rather than relying on text alone.
- If no screenshots are present, the existing text-only analysis flow continues to work without regression.
- Automated or manual validation demonstrates that issues with screenshot-only or screenshot-heavy context produce materially more complete ticket output than before.

Technical Context & Logs

- The current issue reports that Dev-Assist does not use screenshots attached to GitLab issues.
- Prior clarification confirms that screenshot support must apply to attachments on the main issue and to images added later in comments.
- Prior clarification confirms that screenshot findings should surface in the generated ticket text.
- The repository summary shows that GitLab issue processing is coordinated through `src/services/processing/*`, AI integration lives under `src/services/ai/*`, and GitLab access/parsing lives under `src/services/gitlab/*`.
- The repository summary states that Dev-Assist processes GitLab issue and note webhook content and uses the `opencode` CLI with the `dev-assist-analyzer` agent for real analysis.
- Assumption: screenshot handling may depend on whether the active AI/runtime path can receive image inputs or extracted image context in addition to text.

Proposed Solution

- Extend issue and comment context collection so screenshot attachments and embedded images from the main issue and subsequent comments are detected and included in the analysis input.
- Normalize the collected screenshot references and associated metadata into the issue context passed through the processing pipeline.
- Update the AI analysis flow so screenshot content is available alongside textual issue content, using direct image-capable input when supported or a fallback representation when necessary.
- Adjust the analysis instructions or formatting logic so relevant screenshot findings are explicitly reflected in the generated ticket description, acceptance criteria, or technical context when applicable.
- Add automated tests covering screenshots on the main issue, screenshots in comments, mixed text-plus-image issues, and the no-screenshot path to prevent regressions.

Edited just now by dev-assist



 Add design  Create merge request

Edit





Assignee	Edit
 Werner, Nico	
Labels	Edit
None	
Milestone	Edit
None	
Dates	Edit
Start: None	
Due: None	
Time tracking	+
Add an estimate or time spent.	
2 Participants	
 	

Abbildung 5.4.: Dasselbe Issue nach `@dev-assist publish`. Titel und Beschreibung enthalten den gespeicherten Entwurf. Der Screenshot stammt aus dem dokumentierten Lauf zu Abbildung 5.1.

Tabelle 5.1: Leistungsstand der Kernfunktionen im untersuchten Prototyp

Kernfunktion	Umgesetzter Stand
Webhook-Annahme	Issue- und Note-Ereignisse werden geparkt; akzeptierte Lieferungen erhalten HTTP 202, die Analyse läuft danach.
Kontextaufbereitung	Aktuelles Issue, Notes und optional begrenzter Repository-Kontext werden vor der Analyse zusammengeführt.
Modellgrenze	OpenCode oder Mock liefern eine Antwort; nur das JSON-Schema mit sechs Feldern wird akzeptiert.
Rückfrage oder Vorschlag	Mindestens zwei substantielle offene Fragen erzeugen einen Klärungskommentar, sonst einen strukturierten Entwurf.
Speicherung und Publish	Der analysierte Stand wird lokal gespeichert; <code>@dev-assist publish</code> übernimmt ihn ohne neuen Modelllauf.
Nachweisbare Teilprüfungen	Parser, Schema, Formatter und ausgewählte Routen sind durch 60 Tests abgesichert.

Dieser Kern ist lauffähig und lokal ohne reales Modell testbar. Die folgenden Abschnitte zeigen an konkreten Code-Stellen, wie Webhook, Analyse und Publish ineinandergreifen.

5.2. Projektaufbau

Tabelle 5.2 gibt die Übersicht. Der Dienst ist eine Node.js-/TypeScript-Anwendung. `npm run dev` startet den Entwicklungsserver, `npm run build` kompiliert nach `dist/`, `npm test` führt die Testsuite aus. Node.js stellt die Laufzeit bereit, Express das Routing, TypeScript die Prüfung interner Datentypen beim Übersetzen (vgl. Node.js Documentation, o. J.a, o. S.; Express Documentation, o. J., o. S.; TypeScript Documentation, o. J., o. S.). Externe Lieferungen und Modellantworten prüft TypeScript zur Laufzeit nicht. Deshalb bleiben Einlesen und Schema-Prüfung erforderlich.

Tabelle 5.2: Projektbereiche und umgesetzte Verantwortlichkeiten

Projektbereich	Umgesetzte Verantwortung
src/routes/	Health-Check, Webhook-Eingang sowie manuelle Process-/Publish-Endpunkte
src/services/gitlab/	Signatur, Einlesen der Lieferung, Mention, Kommando, GitLab-Client und Löschen der Dev-Assist-Kommentare
src/services/ai/	Prompt, Mock-/OpenCode-Aufruf, Schema, Rückfragen und Markdown-Formatierung
src/services/context/, src/services/ processing/ tests/	Persistenz sowie Orchestrierung der Analyse- und Publish-Phase
	Automatisierte Prüfung ausgewählter Anwendungsteile ohne Sprachmodell sowie von Grenzfällen

src/server.ts startet die Anwendung. src/app.ts registriert Middleware und Routen. Die OpenCode-Agenten liegen in opencode.json. AI_PROVIDER=mock erlaubt einen wiederholbaren Betrieb ohne externes Modell. GitLab-Zugriffe erfolgen über glab oder REST. Signaturprüfung, Mention-Zeichenkette und Speicherpfad steuert die Konfiguration über Umgebungsvariablen.

5.3. Webhook, Aktivierung und Kommando

Ein GitLab-Webhook trifft auf /webhooks/gitlab/issues. Die Route prüft die Signatur, liest die Lieferung in ein internes Ereignis und entscheidet, ob der Text @dev-assist enthält. Eigene Bot-Kommentare und bereits erzeugte Dev-Assist-Antworten werden ignoriert. Danach trennt Listing 5.1 die schnelle HTTP-Antwort von der eigentlichen Arbeit: GitLab erhält Status 202, Analyse oder Publish laufen danach im Hintergrund.

```
(async () => {
  try {
    if (parsed.command === 'publish') {
      await publishIssue(parsed.projectId, parsed.issueId);
    } else {
      await processFromWebhook(parsed);
    }
  } catch (e) {
```

```
    logger.error('Async processing failed', { error: e.message });
  }
})();
res.status(202).json({ accepted: true, command: parsed.command });
```

Listing 5.1: Webhook-Route: Annahme mit HTTP 202, Analyse oder Publish danach
(gitlabWebhooks.ts)

Listing 5.2 erkennt die Aktivierung. Die Funktion prüft, ob die Mention irgendwo im übergebenen Text vorkommt. Bei Issue-Ereignissen wertet der Parser Titel und Beschreibung aus, bei Kommentar-Ereignissen nur den aktuellen Kommentar. So startet eine neue Analyse nicht bei jedem Folgekomentar, sondern nur wenn @dev-assist erneut gesetzt ist.

```
function hasMention(text: string | undefined | null): boolean {
  const t = String(text || '').trim();
  if (!t) return false;
  return mentionPattern.test(t);
}
```

Listing 5.2: Aktivierung: die Mention wird im übergebenen Text gesucht (mention.ts)

Listing 5.3 trennt die beiden Kommandos. Nach dem Entfernen der Mention entscheidet das restliche erste Wort: publish startet die Übernahme, jeder andere Rest startet die Analyse.

```
function parseDevAssistCommand(text: string): DevAssistCommand {
  const rest = mentionGate.stripMention(text).toLowerCase();
  if (rest.startsWith('publish')) return 'publish';
  return 'process';
}
```

Listing 5.3: Kommandotrennung nach der Mention (commands.ts)

5.4. Analyse, Schema und sichtbare Antwort

Der Processor lädt nach Möglichkeit das aktuelle Issue, die Kommentare und eine begrenzte Repository-Zusammenfassung. Scheitert der Abruf, bleibt der Webhook-Text als Fallback. Feste Analyseanweisungen und Ticketdaten gehen getrennt an den Analy-

seagenten. OpenCode oder der Mock-Provider liefern eine Antwort. Listing 5.4 definiert, welche Form akzeptiert wird: genau sechs Felder, Listen als Zeichenketten. Unbekannte Felder oder falsche Typen führen zum Abbruch. Die Prüfung betrifft die Struktur, nicht den fachlichen Inhalt.

```
const REQUIRED_FIELDS = [  
  'title',  
  'description',  
  'acceptanceCriteria',  
  'technicalContext',  
  'proposedSolution',  
  'openQuestions',  
] as const;
```

Listing 5.4: Ausgabeschema der Modellantwort (schema.ts)

Listing 5.5 entscheidet, was GitLab zeigt. Offene Fragen mit mehr als fünf Zeichen gelten als substanziell. Ab zwei solchen Fragen erscheint der Klärungskommentar aus Abbildung 5.2, sonst der vollständige Entwurf aus Abbildung 5.3. In beiden Fällen schreibt der Processor denselben vollständigen Analysekontext lokal nach context.md. Dieser Stand ist die Grundlage für einen späteren Publish.

```
const openQs = (analysis.openQuestions || []).  
  .filter((q) => q && q.trim().length > 5);  
const needsClarification = openQs.length >= 2;  
  
if (needsClarification) {  
  commentToPost = renderClarificationComment(analysis);  
} else {  
  commentToPost = fullContext;  
}  
  
await gitlab.createNote(projectId, issueId, commentToPost);  
await writeContextFile(projectId, issueId, fullContext, metadata);
```

Listing 5.5: Rückfrage oder Entwurf im Issue, vollständiger Stand in der Kontextdatei (processor.ts)

5.5. Publish

@dev-assist publish löst keinen neuen Modelllauf aus. Listing 5.6 liest den gespeicherten Stand, entfernt Dev-Assist-Kommentare und schreibt Titel sowie Beschreibung ins Issue. Genau das zeigt Abbildung 5.4: der Entwurf steht im Ticket, die Arbeitskommentare sind entfernt.

```
const fullMarkdown = await readContextFile(projectId, issueId);
const notes = await gitlab.listNotes(projectId, issueId);
const deletable = filterDeletableNotes(notes);
for (const n of deletable) {
  await gitlab.deleteNote(projectId, issueId, n.id);
}
await gitlab.updateIssue(projectId, issueId, baseUpdates);
```

Listing 5.6: Publish: gespeicherten Stand lesen, Kommentare entfernen, Issue aktualisieren (publisher.ts)

Bilder und Screenshots werden nicht heruntergeladen oder inhaltlich analysiert. Die Repository-Zusammenfassung bleibt auf Metadaten, Sprachen und wenige Dateien begrenzt.

5.6. Lokale Tests

Der Mock-Provider und kleine Parser-, Schema- und Formatterfunktionen ermöglichen lokale Tests ohne Modellkosten. Die vorhandene Suite umfasst 60 Tests in 15 Dateien. Sie prüft vor allem Einlesen, Schema, Darstellung, ausgewählte Routen und die Aufbereitung von Process- und Publish-Daten. Kapitel 6 bewertet, welche Aussagen daraus für die Anforderungen folgen.

Die Implementierung setzt damit den Kern aus Tabelle 5.1 um: Die Modellantwort wird strukturell begrenzt, Analyse und Publish sind getrennt, die Bedienung bleibt in GitLab.

6. Qualitätssicherung und Evaluation

Dieses Kapitel prüft den gebauten Stand gegen die Lösungsidee aus Kapitel 3. Die kurze Antwort vorweg: Der untersuchte Stand lässt sich bauen. Die 60 vorhandenen Tests laufen durch. Der GitLab-Ablauf aus Kapitel 5 ist ausführbar. Mehrere Muss-Anforderungen an Kontrolle sind jedoch nicht erfüllt. Ob die Vorschläge fachlich nutzbar sind, wurde in einer kleinen schriftlichen Expertenbewertung (n=6) an drei Tickets gefragt, nicht in einer Feldstudie gemessen. Der Rest des Kapitels begründet diese Aussage. Der Zielprozess wird hier nicht erneut beschrieben.

Die Untersuchung ist kein Experiment mit Versuchsgruppen und keine Feldstudie (vgl. Easterbrook et al., 2008, S. 285 f. und S. 302 f.).

6.1. Was geprüft wurde – und was nicht

Vier Fragen begrenzen, welche Aussage aus welcher Evidenz folgen darf. Sie sind keine zusätzlichen Forschungsfragen:

1. **EF-1 – Reproduzierbarkeit:** Lässt sich der festgehaltene Programmstand mit den dokumentierten Werkzeugen bauen, und laufen die vorhandenen Tests erfolgreich?
2. **EF-2 – funktionale Übereinstimmung:** Welche Schritte von Aktivierung, Kontext, Analyse, Klärung, Vorschau, Speicherung und Publish sind im Code vorhanden und durch passende Prüfungen belegt?
3. **EF-3 – Sicherheits- und Integritätsgrenzen:** Stimmen Herkunftsprüfung, Eingabeprüfung, Rechte und die Reihenfolge dauerhafter Änderungen mit den Muss-Anforderungen und der GitLab-Dokumentation überein?
4. **EF-4 – Nachweis der Modellwirkung:** Was lässt sich über Richtigkeit, Vollständigkeit, Qualität der Rückfragen und praktischen Nutzen sagen?

Die Prüfung wurde am 31.07.2026 unter Windows mit Node.js v24.11.0 und npm 11.6.1

durchgeführt. Die Textänderungen dieser Arbeit gehören nicht zum geprüften Quellcode. Gebaut und getestet wurde mit `npm run build` und `npm test`.

Tabelle 6.1: Ergebnisse des dokumentierten Build- und Testlaufs

Prüfgröße	Ergebnis
TypeScript-Build	Erfolgreich; keine TypeScript-Fehler
Automatisierte Tests	60 von 60 Tests in 15 Testsuiten bestanden
Teststatus und Laufzeit	Keine fehlgeschlagenen, übersprungenen oder offenen Tests; ca. 0,6 Sekunden
Testabdeckung	Nicht erhoben; Skript und Konfiguration zur Testabdeckung fehlen
Kontinuierliche Integration	Keine GitHub- oder GitLab-CI-Konfiguration im Repository
Ende-zu-Ende gegen GitLab	Nicht als automatisierter Test vorhanden

Tabelle 6.1 dokumentiert einen einzelnen erfolgreichen Lauf. Die Tests prüfen vor allem den Anwendungsteil ohne Sprachmodell: Einlesen, Schema, Darstellung, ausgewählte Laufzeitpfade und Aufbereitung. Es gibt keinen Test, der einen echten GitLab-Webhook über Modellaufruf und Speicherung bis zur Issue-Änderung durchläuft. Die Testzahl darf deshalb weder als Abdeckung noch als Zuverlässigkeitskennzahl gelesen werden.

Statt einer langen Dateiliste fasst Tabelle 6.2 die geprüften Quellen nach ihrer Funktion zusammen.

Tabelle 6.2: Untersuchte projektinterne Artefaktgruppen

Artefaktgruppe	Umfang	Wozu sie dient
Implementierung	src/, opencode.json	Programmwege und Grenzen zwischen Systemen
Testbasis	15 Dateien unter tests/	ausgeführte Fälle mit festgelegten Eingaben
Build und Konfiguration	package.json, tsconfig.json, .env.example	Befehle, Versionen und Sicherheitsstandardwerte
Ziel- und Betriebsbeschreibung	README.md, descriptions/ project_description.txt, AGENTS.md	Sollprozess, Nutzung und Projektkontext

Zusätzlich dient der dokumentierte GitLab-Lauf aus Kapitel 5 als sichtbarer Ablauf. Er ist kein Satz von Testdaten und keine Bewertung mehrerer Tickets. Er macht aber nachvollziehbar, was eine Person im Issue sieht: unvollständiges Ticket, Rückfragen, Antworten, strukturierter Vorschlag und Publish. Der Wortlaut steht in Anhang A.

Eine Anforderung gilt als *nachgewiesen*, wenn Forderung und Umsetzung übereinstimmen und passende Belege vorliegen. *Teilweise nachgewiesen* heißt: Bausteine sind da, der Gesamtschritt bleibt unvollständig. *Nicht erfüllt* gilt, wenn die Implementierung der Muss-Anforderung widerspricht, auch wenn ein Test genau dieses abweichende Verhalten bestätigt. *Nicht untersucht* gilt, wenn keine geeigneten Daten vorliegen. Die Tests wurden nicht nachträglich an die Anforderungen angepasst. Weicht ein bestandener Test von der Anforderung ab, ist das ein eigener Befund.

Für die Kontrollgrenzen gelten drei Regeln:

Ereignisregel: Nur verarbeiten, wenn die Lieferung echt, aktuell und fachlich relevant ist.

Freigaberegeln: Nur veröffentlichen, wenn genau diese Fassung vollständig sichtbar und ausdrücklich freigegeben war.

Änderungsregel: Kommentare erst löschen, wenn die Issue-Aktualisierung erfolgreich war.

6.2. Technische Befunde

6.2.1. Ereignisannahme und Zugriffsschutz

FA-1 fordert `@dev-assist` am Anfang der ersten inhaltlichen Zeile. Die Implementierung akzeptiert dagegen eine Erwähnung im Titel oder mitten im Text. FA-1 ist nicht erfüllt. Issue- und Kommentar-Ereignisse werden vereinheitlicht. Das interne Ereignis enthält jedoch keinen Autor. FA-2 ist deshalb nur teilweise erfüllt. Der Schlüssel gegen Doppelverarbeitung wird gesetzt, bevor geprüft wird, ob das Ereignis fachlich relevant ist. Er verwendet Projekt, Issue, Ereignisart sowie eine Lieferungs-ID oder `no-id`. Unterschiedliche legitime Lieferungen können innerhalb der kurzen Speicherzeit kollidieren. Neustarts und mehrere Instanzen werden nicht abgedeckt.

Die Webhook-Prüfung verwendet `x-gitlab-signature`, `x-gitlab-timestamp` und eine Prüfsumme über `timestamp:body`. Eine Zeitstempelprüfung fehlt. Ist kein Geheimnis konfiguriert, gilt ein Webhook selbst bei aktivierter Signaturpflicht als gültig. Nach den aktuellen Vorgaben von GitLab werden dagegen `webhook-id`, `webhook-timestamp` und `webhook-signature` verwendet; signiert wird `{message_id}.{timestamp}.{body}`, und der Zeitstempel muss aktuell sein (vgl. GitLab Documentation, o. J.a, Abschnitt "Signing tokens"). Die Tests bestätigen damit nur das implementierte Verfahren, nicht die Vorgaben von GitLab. NFA-6 ist nicht erfüllt.

Auch die manuellen Process- und Publish-Wege besitzen keinen eigenen Zugangsschutz. Projekt- und Issue-IDs werden nicht durchgängig als positive Ganzzahlen geprüft. Aus ihnen gebildete Speicherpfade besitzen deshalb keine hinreichend belegte Eingabegrenze. Diese Punkte betreffen den Zugang zum Kernprozess. Eine spätere Schema-Prüfung der Modellantwort kann sie nicht ausgleichen.

6.2.2. Modellgrenze und Ausgabequalität

Die Anwendung akzeptiert nur ein JSON-Objekt mit den sechs Feldern Titel, Beschreibung, Akzeptanzkriterien, technischer Kontext, Lösungsvorschlag und offene Fragen. Falsche Typen und unbekannte Felder werden abgelehnt. Damit ist die strukturelle Antwortgrenze für die geprüften Fälle nachgewiesen. Das fachliche Zielmodell aus Kapitel 3 enthält darüber hinaus Quellenbasis, Risiken und Prüfschritte. Diese Kategorien

haben im kompakten Schema keine eigenen Felder.

Die OpenCode-Auswertung akzeptiert den ersten schemafähigen Kandidaten in Fundreihenfolge. Bekannte Beispiele im Prompt werden nicht ausdrücklich ausgeschlossen. Die Darstellung markiert leere Listen, lässt aber inhaltliche Platzhalter wie `unknown` unverändert. Eine formal gültige Ausgabe kann daher der falsche Kandidat oder fachlich unzureichend sein.

6.2.3. Speicherung und Publish

Im Klärungsfall zeigt GitLab nur Rückfragen. Der Processor speichert dennoch den vollständigen Analysekontext. Der Publisher prüft weder, ob zuvor ein vollständiger Vorschlag sichtbar war, noch ob offene Fragen einer Freigabe entgegenstehen. Die Freigaberegeln, nur den vollständig sichtbaren und freigegebenen Stand zu veröffentlichen, ist damit nicht abgesichert.

Zusätzlich löscht der Publisher ausgewählte Dev-Assist-Kommentare vor der Issue-Aktualisierung, wie Listing 5.6 zeigt. Schlägt diese Aktualisierung fehl, können bereits Klärungskommentare verloren sein. FA-11 ist deshalb nicht erfüllt. Die gewählte Reihenfolge ist keine Komfortfrage. Sie entscheidet, ob der Gesprächsverlauf erhalten bleibt, wenn das Schreiben nach GitLab scheitert.

6.3. Was sich über die Qualität der Vorschläge sagen lässt

EF-4 braucht andere Daten als EF-1 bis EF-3. Die Schema-Prüfung misst, ob eine Antwort die erwarteten Felder besitzt. Sie misst nicht, ob ein Akzeptanzkriterium aus dem Ticket folgt oder ob eine Rückfrage die wichtigste Lücke trifft. Form, inhaltliche Richtigkeit und Nutzen für Umsetzung und Prüfung bleiben getrennt (vgl. Frattini et al., 2023, S. 507–520). Ein gültiges JSON-Objekt kann fachlich falsch oder unbrauchbar sein.

Dafür liegt eine kleine schriftliche Expertenbewertung vor. Sechs Personen, die GitLab-Tickets schreiben oder umsetzen, bewerteten am 18.08.2026 drei echte Beispieltickets: eine Funktionserweiterung (den in Kapitel 5 dokumentierten Lauf), einen Fehlerbericht

und eine technische Aufgabe. Pro Ticket standen Ausgangstext, Rückfragen mit Antworten und der aufbereitete Text untereinander. Sechs Aussagen wurden auf einer Skala von 1 (stimme gar nicht zu) bis 5 (stimme voll zu) mit der Ausweichoption *kann ich nicht beurteilen* gekreuzt. Die Kernfrage lautet, ob der aufbereitete Text insgesamt besser geeignet ist als der Ausgangstext. Ausgewertet wird der Median je Aussage und Ticket. Mittelwerte sind bei $n=6$ nur Hilfsgrößen.

Die Bewertung ist begrenzt. Ausgangstext und aufbereiteter Text sind als solche erkennbar. Die Teilnehmenden kommen aus anderen Projekten, nicht aus dem untersuchten Dev-Assist-Vorhaben. k. b. wurde nicht genutzt. Die Rohwerte stehen in Anhang D.

Was zusätzlich vorliegt, ist der dokumentierte Lauf selbst. Kapitel 7 ordnet Tickettext und Bewertungszahlen gemeinsam ein.

6.4. Überleitung zu den Ergebnissen

EF-1 ist für den dokumentierten Lauf beantwortet. EF-2 bleibt auf ausgewählte Teilwege begrenzt. EF-3 weist Widersprüche zu Muss-Anforderungen und zu den Signaturvorgaben von GitLab nach. EF-4 ist in kleinem Umfang untersucht: sechs Personen bewerteten drei Tickettexte. Das ersetzt keinen Wirkungsnachweis im Feld. Die verdichtete Statusübersicht folgt in Kapitel 7. Ursachen und Folgen behandelt Kapitel 8.

7. Ergebnisse

Drei Aussagen fassen den untersuchten Stand zusammen. Der Zielprozess selbst ist in Kapitel 3 beschrieben, der sichtbare Lauf in Kapitel 5.

Erstens: Der Kernablauf aus Kapitel 3 ist in GitLab ausführbar. Das zeigen die Screenshots in Kapitel 5.

Zweitens: In einer kleinen Expertenbewertung (n=6) lag der Median auf der Frage, ob der aufbereitete Text besser geeignet ist als der Ausgangstext, bei allen drei Tickets bei 5. Das gilt für die bewerteten Texte, nicht für beliebige Modellantworten.

Drittens: Mehrere Kontrollanforderungen sind nicht erfüllt. Die Aktivierung ist zu weit, die Signaturprüfung weicht von GitLab ab, sichtbare Vorschau und gespeicherter Stand können auseinanderlaufen, und Kommentare können vor der Issue-Änderung gelöscht werden.

Ursachen und Folgen behandelt Kapitel 8. Die Forschungsfragen stehen in Kapitel 9.

7.1. Gesamtstand der Anforderungen

Tabelle 7.1 verdichtet die Einzelbefunde aus Kapitel 6. *Teilweise erfüllt* heißt: Ein Teil der Funktion ist vorhanden, der geforderte Umfang oder der Nachweis bleibt begrenzt. *Nicht erfüllt* heißt: Das beobachtete Verhalten widerspricht der Anforderung, oder eine notwendige Kontrolle fehlt. *Nicht untersucht* ist keine Fehlfunktion. Die Kategorie gilt, wenn zur fachlichen Wirkung keine geeigneten Daten vorliegen.

Tabelle 7.1: Konsolidierter Anforderungs- und Nachweisstand des untersuchten Prototyps (eigene Darstellung)

Anforderung oder Grenze	Ergebnis	Verdichtete Aussage
FA-1: Aktivierung	Nicht erfüllt	Eine Erwähnung im Titel oder mitten im Text startet den Dienst, obwohl nur der Anfang der ersten inhaltlichen Zeile erlaubt ist.
FA-2: Autorinformation	Teilweise erfüllt	Die interne Ereignisdarstellung enthält nicht, wer das Ticket oder den Kommentar geschrieben hat.
NFA-6: Authentifizierung	Nicht erfüllt	Die Prüfung ist optional, weicht von den Signaturvorgaben von GitLab ab und enthält keine Zeitstempelprüfung.
Manuelle Routen, Kennungen und Pfade	Nicht erfüllt	Zugriffsschutz und eine hinreichende Prüfung von IDs und daraus gebildeten Speicherpfaden fehlen.
Abgleich von Vorschau und gespeichertem Stand	Nicht erfüllt	Der gespeicherte Publish-Stand kann über den vollständig sichtbaren und bestätigten Vorschlag hinausgehen.
FA-11: Löschreihenfolge	Nicht erfüllt	Kommentare können gelöscht werden, bevor die Issue-Aktualisierung erfolgreich abgeschlossen ist.
NFA-7: Schutz vor Doppelverarbeitung	Teilweise erfüllt	Eine Filterung ist vorhanden. Sie gilt aber nur im laufenden Prozess und hängt von Feldern der GitLab-Lieferung ab, die nicht immer eindeutig sind.
FA-4 bis FA-8: Modellantwort	Form teilweise; Inhalt in kleinem Umfang bewertet	Schema und Verarbeitung sind für ausgewählte Fälle geprüft. Sechs Personen bewerteten drei aufbereitete Texte als klarer und umsetzbarer als den jeweiligen Ausgangstext. Das gilt nicht allgemein.
Kontrollierter Gesamtablauf	Nicht nachgewiesen	Einzelne Schritte sind belegt. Offene Muss-Anforderungen sowie fehlende Gesamtläufe und Wirkungsdaten verhindern den Nachweis des gesamten Ablaufs.

Die Übersicht ist ungleichgewichtig. Die Kette aus Ereignisannahme, Kontext, geprüfter Analyse, Rückfrage oder Vorschlag und getrenntem Publish ist vorhanden. Die Abwei-

chungen sitzen an den Stellen, die den Ablauf kontrollieren sollen: Aktivierung, Herkunft der Meldung, manueller Zugriff und die Bindung zwischen sichtbarem Vorschlag und Veröffentlichung. Bestandene Tests belegen die implementierte Verarbeitung. Sie belegen nicht automatisch, dass die Anforderung erfüllt ist.

7.2. Was der Beispiel-Lauf über die Vorschläge sagt

Die Frage nach der fachlichen Qualität der Vorschläge lässt sich in dieser Arbeit nicht allgemein beantworten. Bewertet werden kann nur der eine dokumentierte Lauf. Es handelt sich um ein eigens angelegtes Demonstrationsissue im untersuchten Projekt, nicht um betriebliche Originaldaten und nicht um eine unabhängige Bewertung. Screenshots stehen in den Abbildungen 5.1 bis 5.4. Der Wortlaut steht in Anhang A.

Ausgangslage war ein unvollständiges Issue. Titel: "Screenshots in the ticket are ignored". Beschreibung: eine Zeile, dass angehängte Screenshots nicht genutzt werden und das besser funktionieren solle. Ziel, Umfang und Erfolgskriterium fehlten.

Dev-Assist stellte zwei Rückfragen: Soll die Screenshot-Unterstützung nur Anhänge am Issue betreffen oder auch Bilder in späteren Kommentaren? Sollen gefundene Bildinhalte im erzeugten Tickettext sichtbar werden oder nur intern in die Analyse einfließen? Beide Fragen treffen die tatsächlichen Lücken. Sie sind einzeln beantwortbar und verlangen keine internen Dateipfade. Die meldende Person antwortete: Unterstützung für Issue und Kommentare; Funde sollen im Tickettext sichtbar werden.

Nach der erneuten Analyse erschien der Vorschlag in Abbildung 5.3. Der veröffentlichte Titel lautete *Use issue screenshots in Dev-Assist analysis*. Die Beschreibung übernimmt die beiden Antworten. Die Akzeptanzkriterien trennen sichtbare Nutzung von Anhängen, Kommentaren, gemischten Tickets und dem Fall ohne Screenshots. Der technische Kontext nennt belegte Hinweise aus Ticket, Klärung und Repository-Zusammenfassung. Der Lösungsvorschlag skizziert fünf Umsetzungsschritte.

Was sich an diesem einen Fall beobachten lässt: Der Assistent hat nicht einfach die Ausgangszeile umformuliert. Er hat fehlende Entscheidungen nachgefragt und die Antworten in Ziel, Kriterien und Lösung übernommen. Das entspricht dem Soll aus Kapitel 3 für Rückfrage und strukturierten Entwurf.

Was sich am Text selbst nicht entscheiden lässt: Ob die Kriterien im Projekt wirklich prüfbar sind. Das letzte Kriterium verlangt selbst wieder eine Validierung und ist damit eher ein Wunsch an die Prüfung als ein beobachtbares Produktergebnis. Mehrere Kriterien wiederholen denselben Umfang in Varianten. Die technischen Hinweise enthalten Dateipfade aus der Repository-Zusammenfassung; sie belegen nicht, dass das Modell den Code verstanden hat. Genau diese Punkte wurden in der Expertenbewertung als Urteil abgefragt, nicht als Messung im Projekt.

Der Lauf macht denselben technischen Befund wie Tabelle 7.1 konkret. Während der sichtbaren Klärung speichert der Processor bereits den vollständigen Analysekontext. Ein `@dev-assist publish` an dieser Stelle könnte mehr übernehmen, als die Person im Issue gesehen und bestätigt hat. Im dokumentierten Lauf wurde erst nach dem vollständigen Vorschlag veröffentlicht. Der Code erzwingt das nicht.

7.3. Expertenbewertung der Ticketttexte

Sechs Personen aus anderen Projekten bewerteten am 18.08.2026 drei echte Beispieltickets. Die Rollen waren Entwicklung (2), Qualitätssicherung (2), Product Owner (1) und DevOps (1). Jede Person kreuzte dieselben sechs Aussagen je Ticket. Die Skala reicht von 1 bis 5. Die Ausweichoption k. b. wurde nicht genutzt. Ticket 1 ist die in Kapitel 5 dokumentierte Funktionserweiterung. Ticket 2 ist ein Fehlerbericht zur Aktivierung. Ticket 3 ist eine technische Aufgabe zum Fehlerkommentar. Der Wortlaut und die Rohwerte stehen in Anhang D.

Tabelle 7.2 zeigt den Median je Aussage. Bei $n=6$ ist der Median die Mitte der geordneten sechs Werte; liegt sie zwischen zwei Zahlen, steht 4,5.

Tabelle 7.2: Median der Expertenbewertung je Aussage und Ticket (n=6; Skala 1–5; eigene Erhebung, 18.08.2026)

Aussage	Ticket 1	Ticket 2	Ticket 3
1 Der aufbereitete Text ist klarer als der Ausgangstext.	5	5	4,5
2 Ziel und Umfang sind verständlich.	5	5	5
3 Die Rückfragen treffen die wichtige Lücke.	5	4	5
4 Die Erfolgskriterien sind prüfbar.	5	5	5
5 Mit diesem Text könnte ich das Ticket umsetzen.	5	4,5	5
6 Insgesamt ist der aufbereitete Text besser als der Ausgangstext.	5	5	5

Von 108 Kreuzungen entfielen 73 auf 5, 34 auf 4 und eine auf 3. Die 3 betrifft Aussage 3 bei Ticket 1 (Rückfragen). Auf Aussage 6 (insgesamt besser) gab es 14-mal 5 und 4-mal 4, nie einen Wert unter 4. Die niedrigsten Mediane liegen bei den Rückfragen von Ticket 2 (4) und bei der Umsetzbarkeit von Ticket 2 (4,5).

Die Zahlen sagen: In den bewerteten Fällen hielten die Personen den aufbereiteten Text für klarer, verständlicher und besser geeignet als den Ausgangstext. Sie sagen nicht, dass Dev-Assist im Alltag Zeit spart oder beliebige Tickets verbessert. Die Verteilung ist nach oben gedrängt. Ein möglicher Grund ist die offensichtliche Gegenüberstellung von knappem Ausgangstext und strukturiertem Entwurf. Die Teilnehmenden kannten das Dev-Assist-Vorhaben nicht aus der eigenen Projektarbeit.

7.4. Zustände eines Vorschlags

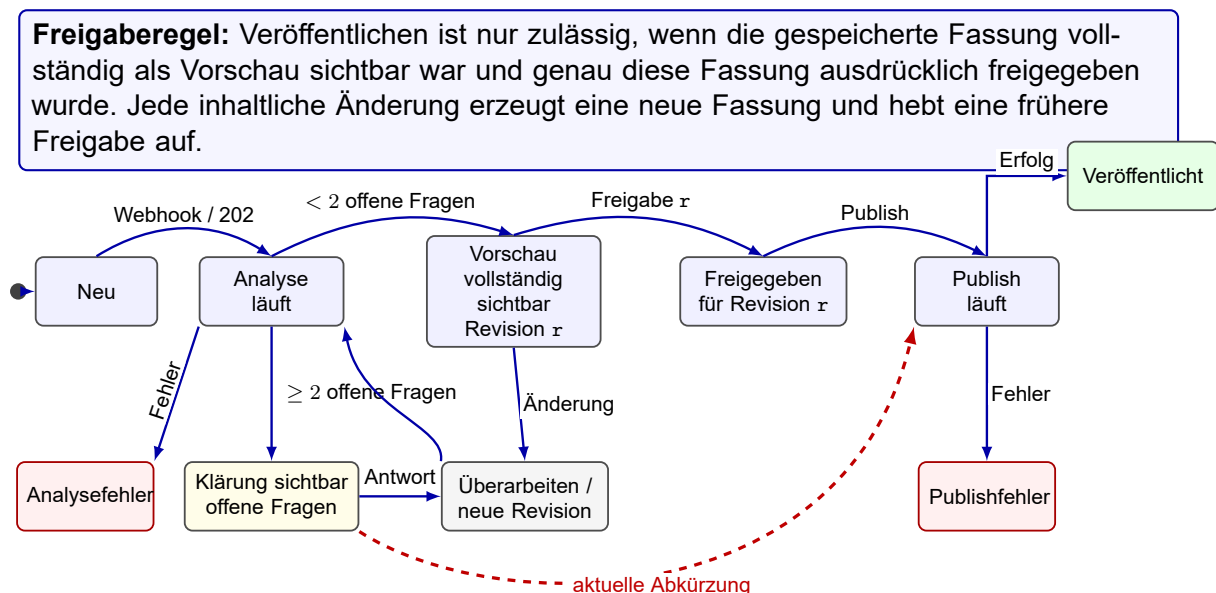
Für die Einordnung der Ergebnisse ist der Zustand eines Vorschlags wichtiger als die bloße Existenz einer Kontextdatei. Ein kontrollierter Ablauf unterscheidet mindestens die Zustände *neu*, *in Analyse*, *in Klärung*, *als vollständige Vorschau sichtbar*, *ausdrücklich freigegeben* sowie *veröffentlicht* oder *Fehler*. Ein Analyseergebnis mit mehreren offenen Fragen darf in die Klärung übergehen. Es darf noch nicht als veröffentlichungs-fähiger Vorschlag gelten.

Die dafür erforderliche Freigaberegeln lautet:

Freigaberegeln:

Die gespeicherte Fassung muss dieselbe sein, die vollständig sichtbar war,
und sie muss ausdrücklich freigegeben worden sein.

Abbildung 7.1 überführt die Regel in ein Zustandsmodell. Blaue Übergänge bilden den erforderlichen kontrollierten Ablauf. Der rot gestrichelte Pfad markiert die im untersuchten Stand mögliche Abkürzung: von der Klärung direkt zur Veröffentlichung, ohne dass der vollständige Entwurf sichtbar war.



Aktuelle Implementierungslücke: Die Anwendung speichert den vollständigen Analysekontext auch dann, wenn GitLab nur Rückfragen zeigt. Im gespeicherten Stand fehlen Status, Versionskennung und Freigabevermerk. Deshalb kann Publish den gespeicherten Vollvorschlag übernehmen, ohne zu prüfen, ob genau diese Fassung sichtbar und freigegeben war.

Abbildung 7.1.: Soll-Zustandsmodell eines Dev-Assist-Vorschlags und Abweichung der aktuellen Implementierung. Die Freigabe muss an dieselbe Fassung gebunden sein, die vollständig in GitLab sichtbar war. Der rot gestrichelte Übergang zeigt die fehlende Prüfung von Status, Fassung und Freigabe. Eigene Darstellung auf Grundlage des untersuchten Quellcodes, Stand 31.07.2026.

Der untersuchte Stand erzwingt die Regel nicht. Während einer sichtbaren Klärung kann ein umfangreicherer Stand gespeichert und später ohne Statusprüfung übernommen werden. Die zeitliche Trennung von Analyse und Publish bildet daher noch keinen kontrollierten Prozess mit menschlicher Prüfung.

Auch Fehler müssen in diesem Modell einen sicheren Zustand ergeben. Scheitert die Issue-Aktualisierung, müssen Issue, Vorschau und Diskussion unverändert nachvollziehbar bleiben. Erst nach bestätigter Kernänderung darf das Löschen der Dev-Assist-Kommentare folgen.

7.5. Reichweite des Ergebnisses

Der Nachweis endet bei den untersuchten Anwendungsteilen ohne Sprachmodell, einem dokumentierten GitLab-Lauf und einer Bewertung von drei Tickettexten durch sechs Personen. Reale Gesamtläufe gegen den Webhook, Lastverhalten und Nutzung über längere Zeit im Team wurden nicht geprüft. Zeitersparnis oder Fehlerreduktion lassen sich daher nicht ableiten. Der Prototyp ist ein technischer Machbarkeitsnachweis ausgewählter Bausteine mit einem kleinen Expertenurteil zur Nutzbarkeit der bewerteten Texte. Er ist kein Nachweis eines kontrollierten oder produktionsreifen Gesamtablaufs.

8. Diskussion

Kapitel 7 hat drei Ergebnisse festgehalten: Der Kernablauf ist in GitLab ausführbar. Sechs Personen bewerteten drei aufbereitete Texte als besser geeignet als den jeweiligen Ausgangstext. Mehrere Kontrollanforderungen sind offen. Diese Diskussion wiederholt die Tabellen nicht. Sie sagt, was aus Lauf und Bewertung folgt – und was nicht.

8.1. Was der Beispiel-Lauf zeigt

Der dokumentierte Lauf beginnt mit einer unbrauchbaren Beschreibung und endet mit einem gegliederten Issue. Dazwischen stehen zwei Rückfragen und zwei kurze Antworten. Das ist genau der Anwendungsfall, den Kapitel 3 als Zielprozess beschreibt: fehlende Angaben sichtbar machen, Antworten einsammeln, erst danach einen Entwurf anbieten.

An diesem einen Ticket lässt sich mehr sagen als “es gibt sechs Felder”. Die Rückfragen betrafen Umfang und sichtbare Verwendung, nicht interne Dateipfade. Die gegebenen Antworten stehen nach Publish in Beschreibung und Kriterien. Der Lösungsvorschlag bleibt eine grobe Umsetzungsskizze und gibt sich nicht als fachliche Wahrheit aus. Das entspricht der in Kapitel 2 gezogenen Grenze: Ein Modell kann strukturieren und nachfragen. Es ersetzt keine Abstimmung (vgl. K C et al., 2025, S. 104–116; Lucassen et al., 2016, S. 383 f. und S. 386 f.).

Zugleich zeigt derselbe Text, warum ein gültiges Schema kein Qualitätsurteil ist. Ein Kriterium verlangt selbst wieder eine Validierung. Mehrere Kriterien sagen denselben Umfang mit anderen Worten. Technische Hinweise stammen teilweise aus der Repository-Zusammenfassung, nicht aus einem geprüften Codeverständnis. Ein wohlgeformtes Feld kann fachlich dünn sein.

Der Lauf wurde vom Autor des Prototyps angelegt. Die meldende Person wusste, welche Antworten der Assistent braucht. Das begünstigt einen sauberen Verlauf. Die Expertenbewertung fragt genau dieses Urteil: Sechs andere Personen kreuzten für Ticket 1 auf Aussage 5 (umsetzbar) vier- oder fünfmal zu. Ein 3 gab es nur bei den

Rückfragen. Das stützt den Einzelfall, ohne ihn von der Anlage durch den Autor zu lösen.

8.2. Warum die Kontrolllücken in genau diesem Lauf zählen

Im dokumentierten Lauf wurde erst nach dem vollständigen Vorschlag veröffentlicht. Der Code erlaubt etwas anderes. Während GitLab nur die Rückfragen aus Abbildung 5.2 zeigt, liegt lokal bereits der vollständige Analysekontext. Ein verfrühtes `@dev-assist publish` könnte diesen Stand ins Issue schreiben, ohne dass jemand den Entwurf aus Abbildung 5.3 gesehen hat.

Die übrigen Lücken haben dieselbe Art von Wirkung. Eine zu weite Aktivierung startet Analysen, die niemand angefordert hat. Eine Signaturprüfung, die ohne Geheimnis durchläuft, akzeptiert fremde Lieferungen. Das vorzeitige Löschen der Kommentare kann ausgerechnet den Klärungsverlauf entfernen, wenn die Issue-Aktualisierung scheitert. Diese Stellen entscheiden, ob der sichtbare GitLab-Ablauf kontrolliert bleibt. Ein besseres Modell allein schließt sie nicht.

8.3. Was die Expertenbewertung zeigt – und was nicht

Die Befragung schließt die offene Lücke nur teilweise. Sie liefert Urteile von sechs Personen an drei Fällen, darunter dem dokumentierten Lauf. Der Median von 5 auf Aussage 6 bedeutet: Im direkten Vergleich wirkte der strukturierte Text geeigneter als die einzeilige Ausgangsmeldung. Das trifft die Frage, ob der Entwurf nur Text umsortiert. Für die bewerteten Fälle sagten die Personen nein.

Zugleich bleiben die Grenzen, die Kapitel 6 benennt. In der Gegenüberstellung waren Ausgangs- und aufbereiteter Text als solche erkennbar. Ein knapper Ausgangstext neben einem gegliederten Entwurf begünstigt hohe Werte. Alle drei Tickets stammen aus dem untersuchten Projekt. Die Teilnehmenden kommen aus anderen Projekten und kannten den Assistenten nicht aus der eigenen Arbeit. Der Median von 5 bleibt trotzdem wenig trennscharf, weil die Skala nach oben gedrängt ist. Die einzige 3 betrifft die Rückfragen von Ticket 1: Eine Person (Product Owner) sah die entscheidende Lücke dort nicht getroffen. Das passt zur Diskussion des Laufs: Die Fragen sind beantwort-

bar, ersetzen aber keine fachliche Prüfung, ob genau diese Scope-Entscheidung die wichtigste war.

Die Befragung misst keine Zeit. Sie vergleicht Dev-Assist nicht mit einer Issue-Vorlage oder einer manuellen Aufbereitung. Issue-Vorlage, manuelle Pflege und der Assistent bleiben deshalb Alternativen ohne Wirkungsnachweis. Der dokumentierte Lauf plus die sechs Bögen dürfen als begrenztes Nutzbarkeitsurteil gelesen werden, nicht als Beleg für Zeitgewinn oder bessere Tickets im Team.

8.4. Grenzen, die aus diesem Material folgen

Build und Schemaerfüllung messen technische Teilfunktionen. Die Expertenbewertung misst ein Urteil über drei Texte, nicht die Umsetzbarkeit im laufenden Projekt. Der untersuchte Stand trägt keine Erfolgsquote und keine Aussage über andere Organisationen. Eine formale Prüfung verhindert weder erfundene Aussagen noch kritische Auslassungen. Tickettexte bleiben Daten, keine Befehle an das Modell. Der getrennte Publish-Schritt schützt nur, wenn genau die gezeigte Fassung freigegeben wird.

Übertragbar ist das Muster aus Ereignisfilter, begrenztem Kontext, geprüfter Modellantwort, sichtbarer Vorschau und ausdrücklicher Freigabe. Die konkreten Webhook-Felder und die Bedeutung von Publish sind GitLab-spezifisch.

8.5. Diskussionsfazit

Der dokumentierte Lauf zeigt, dass Rückfragen und ein gegliederter Entwurf im Issue entstehen können. Sechs Personen bewerteten drei echte Beispieltickets als besser geeignet als den jeweiligen Ausgangstext. Das belegt nicht die fachliche Qualität beliebiger Vorschläge und nicht den kontrollierten Gesamtablauf. Kapitel 9 beantwortet die Forschungsfragen auf dieser Grundlage.

9. Fazit und Ausblick

Die Arbeit untersuchte, welche Angaben ein umsetzbares GitLab-Ticket enthalten muss und wie ein KI-Assistent kontrolliert in GitLab eingebunden werden kann. Dazu entstanden ein fachliches Qualitätsmodell und der TypeScript-/Express-Prototyp Dev-Assist. Der dokumentierte GitLab-Lauf zeigt den Ablauf im Issue. Der Abgleich mit Anforderungen, GitLab-Dokumentation und Tests zeigt, welche Kontrollen der untersuchte Stand hält und welche er verfehlt.

9.1. Beantwortung der Forschungsfragen

9.1.1. Erforderliche Informationen eines Tickets

Die erste Forschungsfrage lautet: *Welche Informationen muss ein GitLab-Ticket enthalten, damit Entwicklerinnen und Entwickler es nachvollziehbar umsetzen und überprüfen können?* Die Untersuchung ergibt dieselben fünf Informationsgruppen wie Tabelle 3.1:

1. **Ziel und Gegenstand:** Titel und Zielbeschreibung benennen Änderung oder Störung und den angestrebten Nutzen.
2. **Kontext und Scope:** Rollen, Systeme, Voraussetzungen, Randbedingungen und Nicht-Ziele grenzen den Auftrag ein.
3. **Verhalten und Kriterien:** Soll- und Ist-Zustand müssen unterscheidbar sein. Fehlerberichte brauchen Reproduktionsschritte. Beobachtbare Akzeptanzkriterien machen das Ergebnis überprüfbar.
4. **Evidenz und Hinweise:** Logs, Fehlermeldungen und Repository-Hinweise ergänzen die Beschreibung mit nachvollziehbarer Herkunft.
5. **Unsicherheit:** Offene Entscheidungen, Widersprüche und fehlende Angaben bleiben erkennbar und führen zu konkreten Rückfragen.

Die Begründung der Kategorien steht in Kapitel 3. Entscheidend sind Relevanz, Nach-

vollziehbarkeit und Prüfbarkeit, nicht Textmenge. Im dokumentierten Lauf fehlten Ziel, Umfang und Erfolgskriterium. Genau dazu entstanden die Rückfragen. Der Lösungsvorschlag des Prototyps bleibt ein veränderbarer Entwurf und keine Quelle fachlicher Wahrheit (vgl. Bettenburg et al., 2008, S. 308 f.; Lucassen et al., 2016, S. 386 f.; K C et al., 2025, S. 104–116).

9.1.2. Kontrollierte Integration eines KI-Assistenten

Die zweite Forschungsfrage lautet: *Wie kann ein KI-gestützter Assistent technisch so in einen GitLab-basierten Entwicklungsworkflow integriert werden, dass Analyse, Rückfragen, Vorschläge und Freigabe kontrolliert ablaufen?* Aus Architektur, Implementierung und Evaluation ergeben sich sechs notwendige Kontrollen:

1. **Ereignisannahme:** Herkunft, Zeitstempel, Typ und ID werden geprüft. Bot-Nachrichten werden gefiltert. Nur eine gesetzte Aktivierung und keine Doppelverarbeitung dürfen den Lauf starten.
2. **Kontextaufbereitung:** Die Anwendung wählt und begrenzt Issue-, Kommentar- und Repository-Daten und behandelt sie als nicht vertrauenswürdig.
3. **Trennung vom Modell:** Das Modell erhält nur den vorbereiteten Kontext und keine Rechte, GitLab selbst zu ändern.
4. **Antwortgrenze:** Einlesen, festes Schema und Auswahl der verwendeten Antwort begrenzen die Modellantwort nach festen Regeln.
5. **Sichtbarer Zwischenstand:** Nur die vollständig sichtbare und danach freigegebene Fassung ist veröffentlichbar.
6. **Fehlersichere Änderung:** Ein autorisierter Publish-Schritt ändert zuerst das Issue. Das anschließende Löschen von Dev-Assist-Kommentaren darf den Ausgangsstand bei Fehlern nicht zerstören.

Kontrolle entsteht durch die Anwendung mit festen Regeln, nicht durch den Prompt allein. Der Prototyp zeigt wesentliche Bausteine im GitLab-Issue. Tabelle 7.1 weist jedoch offene Muss-Anforderungen an Sicherheit und korrekte Änderungsreihenfolge aus. Das

Architekturprinzip ist als Zielmodell tragfähig. Der kontrollierte Gesamtablauf ist im untersuchten Stand nicht nachgewiesen. Fachliche Ticketqualität und technische Kontrolle des Ablaufs sind gemeinsam erforderlich.

9.2. Wissenschaftlicher und praktischer Beitrag

Die Arbeit leistet drei Beiträge: erstens ein aus der Literatur abgeleitetes Qualitätsmodell, zweitens ein Architekturmodell mit überprüfbaren Übergängen zwischen Ereignis, Modellantwort, Vorschau und Issue-Änderung und drittens eine ausführbare Referenzimplementierung mit dokumentiertem GitLab-Lauf, Nachweisstand und einer kleinen Expertenbewertung der Ticketttexte. Formale Struktur, Faktentreue und praktische Nutzbarkeit werden dabei getrennt. Die Freigabe genau der gezeigten Fassung macht eine häufig übersehene Anforderung sichtbar: Es darf nicht mehr veröffentlicht werden, als die Person im Issue gesehen hat.

Die Aussagekraft bleibt auf das untersuchte Projekt, einen Demonstrationslauf und sechs bewertende Personen begrenzt. Messwerte für Zeitgewinn oder Qualitätssteigerung im Feld sind nicht Gegenstand des Nachweises.

9.3. Priorisierter Ausblick

Die weitere Arbeit muss einer sachlichen Reihenfolge folgen. Zuerst ist die Übereinstimmung mit der GitLab-Dokumentation herzustellen. Die kleine Expertenbewertung ersetzt diese Reihenfolge nicht: Sie bewertet Texte, nicht den unsicheren Eingang. Eine belastbare fachliche Prüfung bräuchte mehr Fälle, eine Bewertung ohne Herkunftskennzeichnung und einen Vergleich mit manueller Aufbereitung. Erst auf dieser Grundlage lässt sich praktischer Nutzen bewerten. Tabelle 9.1 verdichtet diese Reihenfolge.

Tabelle 9.1: Priorisierte Roadmap vom Machbarkeitsnachweis zur belastbaren Evaluation

Stufe	Ziel und Maßnahmen	Eintritts- oder Erfolgskriterium
1: Schnittstellen- und Prozesssicherheit	Signaturvorgaben von GitLab, manuelle Zugänge, IDs, Pfade, Freigabe der gezeigten Fassung, Publish-Reihenfolge und Schutz vor Doppelverarbeitung absichern	Positive und negative Integrationstests belegen sichere Eingangs-, Freigabe- und Fehlerpfade
2: Fachliche Bewertung	Die vorhandene Expertenbewertung (n=6, drei echte Beispieltickets) um weitere Fälle, eine Bewertung ohne Herkunftskennzeichnung und einen Vergleich mit manueller Aufbereitung erweitern; Korrekturaufwand messen	Ob Aussagen belegt sind, ob der Kontext ausreicht und ob der Entwurf umsetzbar ist, bleibt vorab messbar festgelegt; die aktuelle Median-Tabelle ist der Ausgangspunkt, nicht der Abschluss
3: Betrieb und Erweiterung	Auftragswarteschlange, Überwachung und Betrieb mit mehreren Instanzen; danach Anhänge, Profile und weitere Ereignisse	Der Mindeststandard bleibt unter Last und Teilausfällen erhalten; Erweiterungen besitzen eigene Sicherheits- und Wiederholungsprüfungen

Die Reihenfolge lautet: Übereinstimmung mit der GitLab-Dokumentation vor fachlicher Qualität und diese vor praktischem Nutzen im Alltag. Andernfalls lassen sich Modell- und Prozessfehler nicht trennen.

9.4. Schlussfolgerung

Generative Modelle können Ticketinhalte strukturieren. Im dokumentierten GitLab-Lauf entstanden gezielte Rückfragen und ein gegliederter Entwurf. Sechs Personen bewerteten drei echte Beispieltickets als besser geeignet als den jeweiligen Ausgangstext. Die fachliche Eignung solcher Vorschläge im Allgemeinen ist damit nicht gezeigt. Verantwortlichkeit und Änderungsrecht verbleiben bei der Anwendung, die nach festen Regeln arbeitet und nur die gezeigte Fassung freigibt. Die Roadmap übersetzt diese Trennung in überprüfbare nächste Schritte: zuerst Schnittstellen und Zustände, dann

eine belastbarere fachliche Prüfung und anschließend die Wirkung im Betrieb.

Quellen

Bashir, Sarmad; Ferrari, Alessio; Khan, Abbas; Strandberg, Per Erik; Haider, Zulqarnain; Saadatmand, Mehrdad; Bohlin, Markus 2025. „Requirements Ambiguity Detection and Explanation with LLMs: An Industrial Study“, in *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, S. 620–631. IEEE. <https://doi.org/10.1109/ICSME64153.2025.00063>.

Bass, Len; Clements, Paul; Kazman, Rick 2021. *Software Architecture in Practice*. 4. Aufl. Boston: Addison-Wesley.

Bettenburg, Nicolas; Just, Sascha; Schröter, Adrian; Weiss, Cathrin; Premraj, Rahul; Zimmermann, Thomas 2008. „What Makes a Good Bug Report?“, in *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, S. 308–318. New York: ACM. <https://doi.org/10.1145/1453101.1453146>.

Bo, Lili; Ji, Wangjie; Sun, Xiaobing; Zhang, Ting; Wu, Xiaoxue; Wei, Ying 2024. „ChatBR: Automated Assessment and Improvement of Bug Report Quality Using ChatGPT“, in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, S. 1472–1483. New York: ACM. <https://doi.org/10.1145/3691620.3695518>.

Breu, Silvia; Premraj, Rahul; Sillito, Jonathan; Zimmermann, Thomas 2010. „Information Needs in Bug Reports: Improving Cooperation Between Developers and Users“, in *Proceedings of the 2010 ACM Conference on Computer Supported Cooperative Work*, S. 301–310. New York: ACM. <https://doi.org/10.1145/1718918.1718973>.

Brown, Tom B.; Mann, Benjamin; Ryder, Nick; Subbiah, Melanie; Kaplan, Jared; Dhariwal, Prafulla et al. 2020. „Language Models Are Few-Shot Learners“, in *Advances in Neural Information Processing Systems* 33, S. 1877–1901. <https://arxiv.org/abs/2005.14165>.

Chaparro, Oscar; Lu, Jing; Zampetti, Fiorella; Moreno, Laura; Di Penta, Massimiliano; Marcus, Andrian; Bavota, Gabriele; Ng, Vincent 2017. „Detecting Missing Information in Bug Descriptions“, in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, S. 396–407. New York: ACM. <https://doi.org/10.1145/3106237.3106285>.

Easterbrook, Steve; Singer, Janice; Storey, Margaret-Anne; Damian, Daniela 2008. „Selecting Empirical Methods for Software Engineering Research“, in Shull, Forrest; Singer, Janice; Sjøberg, Dag I. K. (Hrsg.): *Guide to Advanced Empirical Software Engineering*, S. 285–311. London: Springer. https://doi.org/10.1007/978-1-84800-044-5_11.

Express Documentation o. J. *Express: Node.js Web Application Framework*. <https://expressjs.com/> (Zugriff vom 31.07.2026).

Frattini, Julian; Montgomery, Lloyd; Fischbach, Jannik; Mendez, Daniel; Fucci, Davide; Unterkalmsteiner, Michael 2023. „Requirements Quality Research: A Harmonized Theory, Evaluation, and Roadmap“, in *Requirements Engineering* 28, 4, S. 507–520. <https://doi.org/10.1007/s00766-023-00405-y>.

GitLab Documentation o. J.a. *Webhooks*. <https://docs.gitlab.com/user/project/integrations/webhooks/> (Zugriff vom 31.07.2026).

GitLab Documentation o. J.b. *Webhook Events*. https://docs.gitlab.com/user/project/integrations/webhook_events/ (Zugriff vom 31.07.2026).

GitLab Documentation o. J.c. *Issues API*. <https://docs.gitlab.com/api/issues/> (Zugriff vom 31.07.2026).

GitLab Documentation o. J.d. *Notes API*. <https://docs.gitlab.com/api/notes/> (Zugriff vom 31.07.2026).

Hohpe, Gregor; Woolf, Bobby 2003. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Boston: Addison-Wesley. <https://www.enterpriseintegrationpatterns.com/>.

Huang, Lei et al. 2023. „A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions“. arXiv:2311.05232. <https://doi.org/10.48550/arXiv.2311.05232>.

Ibtasham, Md Saleh; Bashir, Sarmad; Abbas, Muhammad; Haider, Zulqarnain; Saadatmand, Mehrdad; Cicchetti, Antonio 2025. „ReqRAG: Enhancing Software Release Management through Retrieval-Augmented LLMs: An Industrial Study“, in Hess, Anne; Susi, Angelo (Hrsg.): *Requirements Engineering: Foundation for Software Quality. 31st*

International Working Conference, REFSQ 2025, Lecture Notes in Computer Science 15588, S. 277–292. Cham: Springer. https://doi.org/10.1007/978-3-031-88531-0_20.

Inayat, Irum; Salim, Siti Salwah; Marczak, Sabrina; Daneva, Maya; Shamshirband, Shahaboddin 2015. „A Systematic Literature Review on Agile Requirements Engineering Practices and Challenges“, in *Computers in Human Behavior* 51, S. 915–929. <https://doi.org/10.1016/j.chb.2014.10.046>.

ISO/IEC/IEEE 2018. *ISO/IEC/IEEE 29148:2018. Systems and Software Engineering – Life Cycle Processes – Requirements Engineering*. Öffentliche Katalogseite. <https://www.iso.org/standard/72089.html> (Zugriff vom 31.07.2026).

K C, Pragyan; Ghandiparsi, Rambod; Herron, Thomas; Heaps, John; Hosseini, Mitra Bokaei 2025. „Demystifying Feature Requests: Leveraging LLMs to Refine Feature Requests in Open-Source Software“, in *2025 IEEE 33rd International Requirements Engineering Conference (RE)*, S. 104–116. IEEE. <https://doi.org/10.1109/RE63999.2025.00020>.

Krishna, Madhava; Gaur, Bhagesh; Verma, Arsh; Jalote, Pankaj 2024. „Using LLMs in Software Requirements Specifications: An Empirical Evaluation“, in *2024 IEEE 32nd International Requirements Engineering Conference (RE)*, S. 475–483. IEEE. <https://doi.org/10.1109/RE59067.2024.00056>.

Liu, Yi et al. 2023. „Prompt Injection Attack against LLM-Integrated Applications“. arXiv:2306.05499. <https://doi.org/10.48550/arXiv.2306.05499>.

Lubos, Sebastian; Felfernig, Alexander; Tran, Thi Ngoc Trang; Garber, Damian; El Mansi, Merfat; Erdeniz, Seda Polat; Le, Viet-Man 2024. „Leveraging LLMs for the Quality Assurance of Software Requirements“, in *2024 IEEE 32nd International Requirements Engineering Conference (RE)*, S. 389–397. IEEE. <https://doi.org/10.1109/RE59067.2024.00046>.

Lucassen, Garm; Dalpiaz, Fabiano; van der Werf, Jan Martijn E. M.; Brinkkemper, Sjaak 2016. „Improving Agile Requirements: The Quality User Story Framework and Tool“, in *Requirements Engineering* 21, 3, S. 383–403. <https://doi.org/10.1007/s00766-016-0250-x>.

Mahbub, Taslim; Dghaym, Dana; Shankarnarayanan, Aadith; Syed, Taufiq; Shapsough, Salsabeel; Zualkernan, Imran 2024. „Can GPT-4 Aid in Detecting Ambiguities, Inconsistencies, and Incompleteness in Requirements Analysis? A Comprehensive Case Study“, in *IEEE Access* 12, S. 171972–171992. <https://doi.org/10.1109/ACCESS.2024.3464242>.

National Institute of Standards and Technology 2024. *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. NIST AI 600-1. Gaithersburg, MD: National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.AI.600-1>.

Node.js Documentation o. J.a. *Node.js*. <https://nodejs.org/en> (Zugriff vom 31.07.2026).

Node.js Documentation o. J.b. *HTTP*. <https://nodejs.org/api/http.html> (Zugriff vom 31.07.2026).

OpenCode Documentation o. J.a. *Agents*. <https://opencode.ai/docs/agents/> (Zugriff vom 31.07.2026).

OpenCode Documentation o. J.b. *CLI*. <https://opencode.ai/docs/cli/> (Zugriff vom 31.07.2026).

OWASP 2025a. *LLM01:2025 Prompt Injection*. OWASP Top 10 for LLM Applications 2025. <https://genai.owasp.org/llmrisk/llm01-prompt-injection/> (Zugriff vom 31.07.2026).

OWASP 2025b. *LLM06:2025 Excessive Agency*. OWASP Top 10 for LLM Applications 2025. <https://genai.owasp.org/llmrisk/llm062025-excessive-agency/> (Zugriff vom 31.07.2026).

Paiva, Guilherme; Canedo, Edna Dias; Rocha Filho, Geraldo Pereira 2026. „From Issue Titles to Requirements: An Empirical Study of Large Language Models and Prompt Engineering Strategies“, in *Requirements Engineering* 31, Artikel 7. <https://doi.org/10.1007/s00766-026-00462-z>.

Saleem, Summra; Asim, Muhammad Nabeel; Van Elst, Ludger; Dengel, Andreas 2025. „Generative Language Models Potential for Requirement Engineering Applications: In-

sights into Current Strengths and Limitations“, in *Complex & Intelligent Systems* 11, 6, Artikel 278. <https://doi.org/10.1007/s40747-024-01707-6>.

Schick, Timo et al. 2023. „Toolformer: Language Models Can Teach Themselves to Use Tools“, in *Advances in Neural Information Processing Systems* 36, S. 68539–68551. <https://arxiv.org/abs/2302.04761>.

Song, Yang; Chaparro, Oscar 2020. „BEE: A Tool for Structuring and Analyzing Bug Reports“, in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, S. 1551–1555. New York: ACM. <https://doi.org/10.1145/3368089.3417928>.

Tizard, James; Devine, Peter; Wang, Hechen; Blincoe, Kelly 2023. „A Software Requirements Ecosystem: Linking Forum, Issue Tracker, and FAQs for Requirements Management“, in *IEEE Transactions on Software Engineering* 49, 4, S. 2381–2393. <https://doi.org/10.1109/TSE.2022.3219458>.

TypeScript Documentation o. J. *The TypeScript Handbook*. <https://www.typescriptlang.org/docs/handbook/intro.html> (Zugriff vom 31.07.2026).

Xi, Zhiheng et al. 2023. „The Rise and Potential of Large Language Model Based Agents: A Survey“. arXiv:2309.07864. <https://doi.org/10.48550/arXiv.2309.07864>.

Yao, Shunyu et al. 2023. „ReAct: Synergizing Reasoning and Acting in Language Models“, in *International Conference on Learning Representations 2023*. https://openreview.net/forum?id=WE_vluYUL-X.

Zhao, Wayne Xin et al. 2023. „A Survey of Large Language Models“. arXiv:2303.18223. <https://doi.org/10.48550/arXiv.2303.18223>.

A. Illustrierter Ticketverlauf

Der folgende Verlauf dokumentiert denselben GitLab-Lauf, den die Abbildungen 5.1 bis 5.4 zeigen. Das Issue wurde im untersuchten Dev-Assist-Projekt angelegt. Es verwendet das Ausgabeschema und die Formatter-Logik des Prototyps. Betriebliche Originaldaten werden nicht verwendet. Der Verlauf belegt den sichtbaren Prozess. Er ist keine Bewertung der Qualität beliebiger Modellantworten. Die englischen Originaltexte bleiben unverändert.

Ausgangsticket

Titel: Screenshots in the ticket are ignored

Beschreibung:

@dev-assist

If an issue has a screenshot attached, Dev-Assist somehow does not use it. That should work better.

Fehlend sind Ziel, Umfang, sichtbares Erfolgskriterium und die Abgrenzung. Nach FA-6 müssen daraus konkrete Rückfragen entstehen.

Sichtbarer Klärungskommentar

Im untersuchten Stand erzeugt der Processor bei mindestens zwei substanziellen offenen Fragen einen Klärungskommentar. Im dokumentierten Lauf lauteten die Fragen:

- Should screenshot support apply only to attachments on the main issue, or also to images added later in comments?
- Should Dev-Assist surface screenshot findings explicitly in the generated ticket text, or use them only internally to improve analysis quality?

Lokal wird in diesem Fall dennoch der vollständige Analysekontext in `context.md` gespeichert. Genau diese Differenz besteht zwischen sichtbarer Klärung und gespeicher-

tem Vollvorschlag.

Antworten im Kommentar

Die meldende Person hat die Fragen im selben Issue beantwortet, erneut mit führendem @dev-assist:

- Screenshot support should apply to attachments and comments.
- The screenshots should surface in the generated ticket.

Strukturierter Vorschlag nach Klärung

Nach der erneuten Analyse entstand der Vorschlagskommentar aus Abbildung 5.3. Nach @dev-assist publish übernahm das Issue den gespeicherten Stand. Der veröffentlichte Titel lautete *Use issue screenshots in Dev-Assist analysis*. Die Beschreibung folgte dem kompakten Schema:

Description. Dev-Assist should include screenshots from both the main issue and later comments when analyzing a GitLab ticket so that visual information is not ignored during ticket generation. The generated ticket should explicitly surface relevant findings from screenshots instead of using them only internally.

Acceptance Criteria. Unter anderem: Screenshots in der Issue-Beschreibung werden in der Analyse verwendet; Screenshots in Kommentaren ebenfalls; Funde erscheinen im erzeugten Tickettext, wenn sie Ziel, Umfang oder Kriterien betreffen; ohne Screenshots bleibt der bisherige Textpfad erhalten.

Technical Context. Die Klärung bestätigt Umfang und sichtbare Verwendung. Die Repository-Zusammenfassung nennt unter anderem `src/services/processing/`, `src/services/ai/` und `src/services/gitlab/` sowie den Agenten `dev-assist-analyzer`. Als Annahme bleibt offen, ob der aktive Analysepfad Bilder direkt oder nur als extrahierten Text entgegennimmt.

Proposed Solution. Fünf grobe Schritte: Screenshot-Anhänge und eingebettete

Bilder in Issue und Kommentaren erfassen, Metadaten in den Analysekontext übernehmen, den Modellaufruf um Bild- oder Fallback-Darstellung erweitern, Funde in der Ausgabe sichtbar machen und Tests für Anhänge, Kommentare, Mischfälle und den Pfad ohne Screenshots ergänzen.

Im untersuchten Stand prüft der Publisher nicht, ob genau diese Fassung vollständig sichtbar und freigegeben war.

B. Dokumentierter Prüfstand

Die folgende Übersicht wiederholt den in Kapitel 6 dokumentierten Lauf. Die Kurzform soll die Prüfung nachvollziehbar machen.

Tabelle B.1: Kurzprotokoll des dokumentierten Build- und Testlaufs

Größe	Angabe
Datum und Umgebung	31.07.2026, Windows, Node.js v24.11.0, npm 11.6.1
Build	npm run build, ohne TypeScript-Fehler
Tests	npm test: 60 von 60 Tests in 15 Suiten bestanden, ca. 0,6 Sekunden
Testabdeckung	nicht konfiguriert
Ende-zu-Ende gegen GitLab	nicht ausgeführt

Die Tests prüfen vor allem Parser, Schema, Formatter, ausgewählte Laufzeitpfade und Aufbereitungslogik. Sie ersetzen weder einen realen Webhook-Lauf noch eine fachliche Bewertung von Modellantworten.

C. Ausgabeschema des Prototyps

Das implementierte Ausgabeschema akzeptiert ausschließlich die folgenden Felder. Unbekannte Felder oder falsche Datentypen werden abgelehnt.

Tabelle C.1: Felder des kompakten Ausgabeschemas

Feld	Typ	Zweck
title	Zeichenkette	prägnanter Issue-Titel
description	Zeichenkettenliste	Ziel, Nutzen, Verhalten und Abgrenzung in Absätzen
acceptanceCriteria	Zeichenkettenliste	beobachtbare Prüfkriterien
technicalContext	Zeichenkettenliste	belegte technische Hinweise, Logs, Annahmen
proposedSolution	Zeichenkettenliste	grobe Umsetzungsschritte ohne fachliche Erfindung
openQuestions	Zeichenkettenliste	unbeantwortete fachliche Fragen

Quellenbasis, Risiken und Validierungsschritte des fachlichen Zielmodells aus Kapitel 3 besitzen im kompakten Ausgabeschema keine eigenen Felder. Sie können nur innerhalb der allgemeinen Listen erscheinen oder entfallen.

D. Expertenbewertung

Die Bewertung fand am 18.08.2026 schriftlich statt. Sechs Personen aus anderen Projekten kreuzten je Ticket sechs Aussagen (1 = stimme gar nicht zu, 5 = stimme voll zu, k. b. = kann ich nicht beurteilen). k. b. wurde nicht genutzt. Die Bögen tragen die Codes T-01 bis T-06, keine Namen.

Die drei Tickets sind echte Beispiele aus dem untersuchten Projekt. Ticket 1 ist der in Kapitel 5 und Anhang A dokumentierte Lauf (Ausgangstitel *Screenshots in the ticket are ignored*, aufbereiteter Titel *Use issue screenshots in Dev-Assist analysis*). Ticket 2 ist der Fehlerbericht *Mentions in the middle of a comment start a run*. Ticket 3 ist die technische Aufgabe *Failed analysis leaves the issue silent*.

Die sechs Aussagen lauten:

1. Der aufbereitete Text ist klarer als der Ausgangstext.
2. Ziel und Umfang sind verständlich.
3. Die Rückfragen treffen die wichtige Lücke.
4. Die Erfolgskriterien sind prüfbar.
5. Mit diesem Text könnte ich das Ticket umsetzen.
6. Insgesamt ist der aufbereitete Text besser als der Ausgangstext.

Tabelle D.1: Rohwerte der Expertenbewertung (1–5; eigene Erhebung, 18.08.2026)

Code	Rolle	T	1	2	3	4	5	6
T-01	Entwicklung	1	5	5	4	5	5	5
T-01	Entwicklung	2	5	4	5	5	4	5
T-01	Entwicklung	3	5	5	5	4	5	5
T-02	Qualitätssicherung	1	4	5	5	4	4	5
T-02	Qualitätssicherung	2	5	5	4	5	5	4
T-02	Qualitätssicherung	3	4	4	5	5	4	5
T-03	Product Owner	1	5	4	3	5	4	5
T-03	Product Owner	2	4	5	4	4	5	4
T-03	Product Owner	3	5	5	4	5	5	5
T-04	Entwicklung	1	5	5	5	5	5	5
T-04	Entwicklung	2	5	5	5	5	4	5
T-04	Entwicklung	3	4	5	5	5	5	4
T-05	Qualitätssicherung	1	4	4	5	4	5	4
T-05	Qualitätssicherung	2	5	4	4	5	4	5
T-05	Qualitätssicherung	3	4	5	4	4	5	5
T-06	DevOps	1	5	5	5	5	5	5
T-06	DevOps	2	5	5	4	5	5	5
T-06	DevOps	3	5	4	5	5	5	5

Spalte T ist die Ticketnummer. Die Spalten 1 bis 6 sind die sechs Aussagen.